
Lógica modal en Lean



UNIVERSIDAD
COMPLUTENSE
MADRID

Jorge Martinena Cepa
Facultad de Ciencias Matemáticas
Universidad Complutense de Madrid

Trabajo de Fin de Grado
Grado en Matemáticas
(Especialización en Ciencias de la Computación)

Septiembre de 2026

Dirigido por Ignacio Fábregas y Óscar Martín

Resumen

Este trabajo presenta la formalización en el asistente de demostración Lean 4 de la lógica modal **K** y de sus dos resultados fundamentales: los teoremas de corrección y completitud respecto a la semántica de Kripke. Frente al estilo conciso de una biblioteca como Mathlib, esta formalización se ha desarrollado con una intención deliberadamente didáctica para hacer visible cada paso del razonamiento lógico. La implementación abarca desde la sintaxis básica hasta la construcción del modelo canónico y una versión del lema de Lindenbaum basada en el lema de Zorn. La memoria expone los fundamentos lógicos y computacionales necesarios sin presuponer conocimientos previos, justificando las decisiones de diseño y las tácticas empleadas, y culmina con un ejemplo ilustrativo de aplicación de la lógica modal.

Tabla de contenidos

Resumen	i
1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	3
1.3. Metodología y plan de trabajo	3
1.4. Estructura de la memoria	4
2. El asistente de demostración Lean	6
2.1. La demostración asistida por ordenador	6
2.2. El fundamento: la teoría de tipos	7
2.2.1. Todo término tiene un tipo	7
2.2.2. Las proposiciones son tipos	8
2.2.3. Qué son los tipos inductivos	9
2.3. Definiciones y enunciados	11
2.4. El modo táctico y el InfoView	13
2.5. Catálogo de tácticas empleadas	15
2.5.1. Tácticas básicas de manipulación de objetivos	15
2.5.2. Construir y descomponer estructuras lógicas	17
2.5.3. Razonamiento clásico	18
2.5.4. Inducción y reescritura	19
2.6. Mathlib, la lógica clásica y el lema de Zorn	21
3. La lógica modal $\mathbf{K}$	23
3.1. ¿Necesidad y posibilidad?	23
3.2. Sintaxis	24
3.3. El sistema axiomático $\mathbf{K}$	25
3.4. Semántica de Kripke	29
3.5. Corrección y completitud: el mapa de la demostración	32

4. La formalización en Lean	37
4.1. La sintaxis	37
4.2. El esquema de tautologías	38
4.3. El sistema axiomático	40
4.4. La semántica de Kripke	42
4.5. El teorema de corrección	43
4.6. El teorema de completitud	46
4.6.1. Deducción a partir de hipótesis	46
4.6.2. Consistencia y conjuntos maximales consistentes	47
4.6.3. El lema de Lindenbaum, vía Zorn	49
4.6.4. El modelo canónico y el lema de encajonado	51
4.6.5. Los lemas de existencia y de verdad	52
4.6.6. Completitud y adecuación	54
5. Hacia la lógica epistémica: el problema de los niños embarrados	57
5.1. El acertijo	57
5.2. El conocimiento como modalidad	59
5.3. El modelo del acertijo	60
5.4. Cómo los anuncios públicos cambian el modelo	63
5.5. El teorema	64
5.6. El anuncio del adulto y el conocimiento común	67
6. Conclusiones	69
6.1. Aprendizaje de Lean, lógica modal y formalización.	69
6.2. Reflexiones sobre Lean y la mecanización de las demostraciones	70
6.3. Mejoras y ampliaciones posibles	71
Bibliografía	72

Capítulo 1

Introducción

1.1. Motivación

Este trabajo se presenta como la combinación de dos intereses muy ligados a las matemáticas que son, en principio, independientes: los asistentes de demostración y la lógica modal.

Los asistentes de demostración

Una demostración matemática es, teóricamente, un objeto completamente preciso: una cadena de pasos en la que cada afirmación se sigue de las anteriores mediante reglas aceptadas. En la práctica, sin embargo, las demostraciones que escribimos rara vez son absolutamente formales. Hay pasos “evidentes” que se omiten, casos “análogos” que no se comprueban o notación que se sobreentiende. En general, este nivel de detalle es el adecuado para hacer matemáticas entre personas, pero deja abierta una pregunta incómoda respecto de la formalización pura: ¿cómo sabemos que no se nos ha escapado nada?

Los asistentes de demostración atacan esta pregunta de frente. Son programas que permiten escribir definiciones, enunciados y demostraciones en un lenguaje formal, y que solo aceptan una demostración cuando cada uno de sus pasos ha sido verificado mecánicamente a partir de los axiomas del sistema y los resultados previamente introducidos y comprobados. La idea no es nueva. Se remonta a proyectos como Automath [6], iniciado en 1967, o Mizar [9], de 1973. Además, en los últimos años ha ganado un impulso enorme. Sistemas como Rocq (anteriormente Coq) o Lean cuentan hoy con comunidades muy activas y con bibliotecas que cubren buena parte del grado en Matemáticas y más allá [15]. En este trabajo se usa Lean 4, por la cantidad de material abierto para su aprendizaje en línea y la cultura de uso que está comenzando a generarse en nuestra Facultad.

En lo personal, los asistentes de demostración, con sus respectivas bibliotecas, despiertan un interés particular por el acercamiento aparente que presentan al ideal de construcción de un “árbol de las matemáticas”. Esto es, la posibilidad de tener, de forma tangible, un grafo de todos los teoremas y resultados matemáticos y cómo se derivan unos de otros. A grandes rasgos y de forma ingenua, esto tendría dos implicaciones muy interesantes en direcciones opuestas: en un sentido descendente, poder generar un árbol de dependencias para cada resultado, teorema, etc; en otro ascendente, la posibilidad de que, el avance tecnológico y de la inteligencia artificial, unidos al desarrollo de la verificación mecánica de resultados, permitan avanzar en un futuro en la demostración de problemas abiertos. Este ideal es, como se ha mencionado antes, atractivo pero ingenuo por varias razones, algunas de las cuales se enfrentarán en el desarrollo de este trabajo y se comentarán en sus conclusiones.

La lógica modal

La lógica modal extiende la lógica proposicional clásica con dos operadores nuevos: $\Box$ y $\Diamond$ que, habitualmente, se leen como “*es necesario*” y “*es posible*” respectivamente. Más adelante veremos que esto no es más que su interpretación clásica, pero existen otras dependiendo de la modalidad que se quiera considerar. Su semántica habitual se define sobre los modelos de Kripke, que son esencialmente grafos dirigidos cuyos nodos, llamados *mundos posibles*, llevan asociada una valoración de las variables proposicionales. El sistema modal más básico es la lógica **K**, que se obtiene añadiendo al cálculo proposicional el axioma K, que distribuye $\Box$ sobre la implicación, y la regla de necesidad, que permite concluir $\Box\varphi$ de un teorema φ . Los dos resultados centrales de la teoría son los teoremas de *corrección* y *completitud*: todo lo demostrable en **K** es válido en todos los modelos de Kripke y, recíprocamente, todo lo válido es demostrable.

La integración de la lógica modal y Lean en este trabajo resulta especialmente apropiada por varios motivos. Primero, es un tema autocontenido: su sintaxis y su cálculo se definen desde cero, sin apoyarse en grandes construcciones previas, lo que permite formalizarlo casi entero en lugar de importar la mitad del trabajo de una biblioteca. Segundo, tiene una dificultad suficiente: la corrección es un ejercicio asequible de inducción, mientras que la completitud, con sus múltiples pasos intermedios, es una demostración no trivial que pone a prueba tanto al asistente como a quien lo maneja. Y tercero, es un tema con recorrido y relevancia en mi rama de especialización: sobre la lógica **K** se construyen las lógicas epistémicas y temporales que se usan en informática para razonar sobre conocimiento y sobre programas.

Hay que señalar que la lógica modal ya tiene formalizaciones con un estilo más sintético, similar al de la biblioteca Mathlib, y, en algún caso, con importante carga de importaciones

de la misma [13, 17]. Este trabajo no pretende competir con esas formalizaciones, sino hacer algo distinto con ellas como referencia lejana: una formalización didáctica, pensada para ser leída. Las demostraciones de Mathlib están optimizadas para ser mantenibles y compactas, y con frecuencia resultan casi ilegibles para quien no es experto. Las expuestas en este trabajo, en cambio, están escritas paso a paso, comentadas, y eligiendo en cada momento la táctica que mejor refleja el razonamiento matemático que se está haciendo, aunque no sea necesariamente lo más óptimo para un asistente de demostración.

1.2. Objetivos

El objetivo general del trabajo es formalizar en Lean 4 la lógica modal **K** y demostrar en este entorno su corrección y completitud respecto de los modelos de Kripke, tal como fija la propuesta del TFG. Este objetivo general se concreta en los siguientes objetivos específicos:

1. Aprender el manejo de Lean 4 como asistente de demostración partiendo de cero: su fundamento en teoría de tipos, su lenguaje de definiciones y su modo táctico.
2. Estudiar la lógica modal **K** y su semántica de Kripke, siguiendo principalmente el texto de Blackburn, de Rijke y Venema [4], hasta la demostración clásica de completitud por modelos canónicos.
3. Formalizar la sintaxis, el sistema axiomático y la semántica, y demostrar formalmente los teoremas de corrección y completitud, junto con los resultados auxiliares que requieren (teorema de deducción, lema de Lindenbaum, lema de verdad, etc.).
4. Como ampliación, explorar una aplicación de la maquinaria anterior al razonamiento epistémico, tomando como hilo conductor el problema de los niños embarrados (*muddy children*).

1.3. Metodología y plan de trabajo

El trabajo se ha desarrollado en tres fases que, en la práctica, se han solapado constantemente. Debe tenerse en cuenta que, para el presente trabajo, se ha tenido que aprender un software desconocido para formalizar un contenido matemático igualmente desconocido.

Fase de aprendizaje. El primer contacto con Lean se realizó a través de materiales introductorios de la comunidad: el *Natural Number Game* [5], el libro *Theorem Proving in Lean 4* [1] y *Mathematics in Lean* [2]. En paralelo se estudió la parte de lógica modal, principalmente los capítulos 1 y 4 de [4], que cubren sintaxis, semántica y la completitud por modelos canónicos.

Fase de formalización. La formalización se construyó de forma incremental, en el mismo orden en que se presenta en el capítulo 4: sintaxis, sistema axiomático, semántica, corrección y, por último, completitud. Cada bloque se validó con ejemplos y algunos lemas que sirvieran de verificación y ejemplificación (por ejemplo, comprobar que las conectivas derivadas reciben el significado semántico esperado) antes de pasar al siguiente. Cabe destacar que hay dos decisiones de diseño que separan nuestra formalización del desarrollo en papel de [4] y que se justifican con detalle en su momento: la noción de deducibilidad desde hipótesis, que se define de forma inductiva, y el lema de Lindenbaum, que se demuestra importando el lema de Zorn, lo que permite eliminar la hipótesis de numerabilidad del lenguaje.

Fase de redacción. La memoria se ha escrito con el código ya estable, lo que ha permitido que cada táctica y cada construcción de Lean que se explica en el capítulo 2 aparezca realmente usada en el capítulo 4, y recíprocamente. Es decir, no se explica nada que no se use, ni se usa nada que no se explique.

Uso de herramientas de inteligencia artificial

Es necesario hacer una aclaración sobre las herramientas empleadas. La inmensa mayoría de las demostraciones se han escrito a mano, con la ayuda del *InfoView* de Lean. En un número reducido de teoremas técnicos se ha recurrido a herramientas de inteligencia artificial, en concreto Claude, como apoyo para encontrar las tácticas adecuadas para el desarrollo de la demostración. En todos los casos, las demostraciones han sido escritas y desarrolladas manualmente, revisadas, comentadas línea a línea y, por supuesto, verificadas por Lean, que es en última instancia quien garantiza su corrección. También se ha usado puntualmente para construir la modelización particular del problema de los *muddy children* del capítulo 5.

1.4. Estructura de la memoria

El resto de la memoria se organiza como sigue:

- El capítulo 2 presenta Lean: qué es un asistente de demostración, en qué teoría de tipos se fundamenta, cómo se escriben definiciones y demostraciones, y un catálogo razonado de todas las tácticas que se emplean en este trabajo.
- El capítulo 3 desarrolla la parte matemática: la sintaxis del lenguaje modal, el sistema **K**, la semántica de Kripke y el mapa de la demostración de completitud que después se formaliza.

- El capítulo 4 es el núcleo del trabajo: recorre la formalización completa, desde el tipo inductivo de las fórmulas hasta el teorema de adecuación, explicando y justificando cada decisión de diseño y cada demostración.
- El capítulo 5 esboza la ampliación hacia la lógica epistémica y el problema de los niños embarrados.
- El capítulo 6, finalmente, recoge las conclusiones y otros comentarios finales.

El código Lean íntegro junto con el código $\text{\LaTeX}$ de esta memoria se pueden encontrar en el siguiente repositorio:

<https://github.com/Jorbol/tfg-logica-modal-lean>

Capítulo 2

El asistente de demostración Lean

Este capítulo presenta Lean con un criterio concreto: explicar lo aprendido en el manejo de la herramienta y lo necesario para la formalización del capítulo 4. Empezamos situando Lean entre los asistentes de demostración (sección 2.1); después describimos su fundamento teórico, la teoría de tipos dependientes, en la medida en que hace falta para entender qué significa que “una demostración es un término” (sección 2.2); a continuación explicamos cómo se escriben definiciones y teoremas (sección 2.3); cómo funciona el modo táctico (sección 2.4); y cerramos con un catálogo razonado de todas las tácticas que se usan en este trabajo (sección 2.5) y con las herramientas de Mathlib de las que dependemos, en particular la lógica clásica y el lema de Zorn (sección 2.6).

2.1. La demostración asistida por ordenador

Un asistente de demostración es un programa con dos componentes bien diferenciadas. Por un lado, un lenguaje formal en el que el usuario escribe definiciones, enunciados y demostraciones. Por otro, un núcleo (*kernel*) que es un verificador pequeño que comprueba, paso a paso, que cada demostración es correcta según las reglas de la lógica subyacente. Todo lo demás (editor, tácticas, automatización...) son capas de comodidad. Pueden ayudar a construir la demostración, pero la única autoridad que la acepta es el núcleo. Esta arquitectura, conocida como criterio de De Bruijn [3], es lo que hace razonable fiarse del sistema. Esto es, para dudar de un teorema verificado hay que dudar de un programa de miles de líneas, no de cientos de miles que forman el resto de la herramienta.

Los primeros sistemas de este tipo, como Automath (1967) [4] y Mizar (1973) [9], demostraron que la idea era viable. Hoy los asistentes más usados en matemáticas son Rocq, Isabelle/HOL y Lean. Lean [7], desarrollado inicialmente por Leonardo de Moura en Microsoft Research y mantenido actualmente por la Lean Focused Research Organisation, es a la vez un asistente de demostración y un lenguaje de programación funcional. En este

trabajo usamos su cuarta versión, Lean 4, junto con su biblioteca matemática Mathlib [15], un proyecto comunitario que reúne, verificadas, buena parte de las matemáticas de grado y posgrado. De Mathlib solo importaremos su infraestructura básica: la teoría elemental de conjuntos, el modo táctico y, en un punto crucial, el lema de Zorn.

En la práctica, trabajar con Lean se parece a mantener un diálogo. El usuario escribe en el editor y una ventana lateral, el *InfoView*, muestra en cada momento el estado de la demostración: qué hipótesis hay disponibles y qué falta por demostrar. Cada instrucción del usuario transforma ese estado, y la demostración termina cuando no queda nada pendiente por probar. Veremos ese mecanismo con detalle en la sección 2.4.

2.2. El fundamento: la teoría de tipos

La lógica sobre la que se construye Lean no es la teoría de conjuntos de Zermelo–Fraenkel, sino una *teoría de tipos dependientes* llamada cálculo de construcciones con tipos inductivos. No necesitamos conocer este cálculo en profundidad, pero sí entender tres de sus ideas, porque aparecen constantemente en la formalización: 1) todo término tiene un tipo, 2) las proposiciones son tipos, y 3) qué son los tipos inductivos.

2.2.1. Todo término tiene un tipo

En teoría de tipos, los objetos básicos son los términos, y cada término tiene asignado un tipo. La notación $x : T$ se lee “ x es un término de tipo T ” y es la afirmación fundamental del sistema; juega un papel análogo al de la pertenencia $x \in A$ en teoría de conjuntos, con una diferencia importante: un término tiene un único tipo, que puede calcularse mecánicamente. Por ejemplo, $(2 : \text{Nat})$ declara que 2 es un número natural, y Nat es a su vez un término cuyo tipo es Type , el “tipo de los tipos” de datos.¹

Los tipos de funciones se escriben con una flecha. Una función de los naturales en los naturales tiene tipo $\text{Nat} \rightarrow \text{Nat}$, y si $f : \text{Nat} \rightarrow \text{Nat}$ y $n : \text{Nat}$, la aplicación se escribe por yuxtaposición, $f\ n$, sin paréntesis. Las funciones de varios argumentos se representan encadenando flechas, que asocian a la derecha: $\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$ es el tipo de las funciones que reciben un natural y devuelven una función $\text{Nat} \rightarrow \text{Nat}$; en la práctica, una función de dos argumentos.

Las funciones anónimas se escriben con la palabra clave `fun`. Se trata de las λ -expresiones del cálculo lambda sobre el que se construye la teoría de tipos de Lean, y de hecho λ es sintaxis igualmente válida y con el mismo significado. Por ejemplo:

¹Para evitar paradojas del estilo de la de Russell, `Type` no puede ser término de sí mismo: Lean organiza los tipos en una jerarquía infinita `Type 0`, `Type 1`, `Type 2`, ... llamada *jerarquía de universos*. En este trabajo no hará falta subir nunca de `Type` (que abrevia `Type 0`) y `Prop`, así que no volveremos a mencionar los universos.

```

1 example : Nat → Nat      := fun n => n + 1
2   -- la función que a cada n le asigna n + 1
3 example : Nat → Nat → Nat := fun n m => n + m
4   -- dos argumentos: abrevia fun n => (fun m => n + m)
5 example : Nat → Nat → Prop := fun _ _ => False
6   -- el guion bajo, un argumento cuyo nombre no interesa

```

La segunda línea muestra, del lado de los términos, el mismo encadenamiento que acabamos de ver en los tipos. La tercera aparecerá tal cual al definir modelos concretos en el capítulo 4, como una relación que no relaciona nada con nada.

La teoría es dependiente porque el tipo del resultado de una función puede depender del valor del argumento. El ejemplo que nos importa es el cuantificador universal. El tipo $\forall x : A, P x$ es el tipo de las funciones que a cada $x : A$ le asignan un término de tipo $P x$. Lo característico es que aquí lo que varía con x no es solo el valor del resultado, como en $A \rightarrow B$, sino su propio tipo: si $f : \forall x : A, P x$ y tomamos x e y distintos, los resultados $f x$ y $f y$ habitan tipos distintos, $P x$ y $P y$. El tipo $A \rightarrow B$ es el caso particular en que P no depende de x . Esta lectura del $\forall$ como un tipo de funciones tiene consecuencias muy prácticas que veremos enseguida.

2.2.2. Las proposiciones son tipos

Junto a `Type`, Lean tiene otro universo llamado `Prop`, el tipo de las proposiciones. Un término $p : \text{Prop}$ es un enunciado (“2 es par”, “ φ es válida”), y la idea central del sistema, conocida como correspondencia de Curry–Howard o *proposiciones como tipos*, es la siguiente:

Una demostración de la proposición p es un término de tipo p .

Es decir, las proposiciones se tratan exactamente igual que los demás tipos, y demostrar p consiste en construir un término $h : p$. Si no existe ningún término de tipo p , la proposición no es demostrable. Bajo esta correspondencia, las conectivas lógicas se convierten en constructores de tipos y su comportamiento queda determinado por el de los tipos correspondientes:

- **Las que son funciones.** Su demostración es un procedimiento que transforma demostraciones en demostraciones, y usarla consiste en aplicarlo.
 - La implicación $p \rightarrow q$ es el tipo de las funciones de p en q : demostrarla es dar una función que convierte demostraciones de p en demostraciones de q . Por eso, si $h : p \rightarrow q$ y $hp : p$, entonces $h hp : q$. El *modus ponens* es la aplicación de funciones.

- El cuantificador $\forall x : A, P\ x$ es el tipo de funciones dependientes descrito antes: demostrarlo es dar un procedimiento que a cada x le asigna una demostración de $P\ x$, y usarlo sobre un elemento concreto a es aplicar la función, $h\ a : P\ a$.
 - La negación $\neg p$ se define como $p \rightarrow \text{False}$, donde `False` es la proposición sin demostraciones (un tipo vacío) y `True` la que tiene una demostración trivial. Demostrar que p es falsa es, pues, dar una función que convierta cualquier hipotética demostración de p en una demostración de lo imposible. Este detalle será importante. Nuestra negación modal $\sim\varphi := \varphi \rightarrow \perp$ imita exactamente esta definición, y esa imitación hará que varios lemas semánticos sean ciertos “por definición”.
- **Las que son pares (o sumas).** Su demostración se construye empaquetando piezas, y usarla consiste en extraerlas.
- La conjunción $p \wedge q$ es el tipo de los pares: una demostración suya es un par $\langle hp, hq \rangle$ formado por una demostración de cada componente, que se recuperan con `.1` y `.2`.
 - El bicondicional $p \leftrightarrow q$ es un par de implicaciones, cuyas componentes se extraen con `.mp` (de izquierda a derecha) y `.mpr` (de derecha a izquierda).
 - El existencial $\exists x : A, P\ x$ se demuestra con un par $\langle a, ha \rangle$: un testigo a y una demostración $ha : P\ a$. Es un par en el que el tipo de la segunda componente depende de la primera.
 - La disyunción $p \vee q$ es la excepción de este grupo: no es un par sino una suma, con dos constructores, `Or.inl hp` y `Or.inr hq`, según qué componente se demuestre. Construir la exige elegir un lado. Usarla, en cambio, obliga a tratar los dos casos posibles.

Los corchetes angulares $\langle \ , \ \rangle$ que acaban de aparecer son el constructor anónimo: una notación uniforme para construir términos de cualquier tipo con un solo constructor. Los tres primeros casos del segundo grupo son instancias de lo mismo, un par, con una o dos componentes que pueden depender entre sí, y por eso los tres se escriben igual; la disyunción, con sus dos constructores, queda fuera. Esta notación aparecerá constantemente, también para los elementos de las estructuras que definamos.

2.2.3. Qué son los tipos inductivos

La tercera idea es el mecanismo con el que se crean tipos nuevos: los tipos inductivos. Un tipo inductivo se define dando una lista de constructores, y sus términos son exactamente

los que se obtienen aplicando esos constructores un número finito de veces. El ejemplo clásico son los números naturales, definidos en la biblioteca de Lean esencialmente así:

```
1 inductive Nat where
2   | zero : Nat          -- el cero es un natural
3   | succ : Nat → Nat    -- el sucesor de un natural es un natural
```

Cada línea que empieza por `|` declara un constructor con su tipo. La definición debe leerse como: los términos de tipo `Nat` son `Nat.zero`, `Nat.succ Nat.zero`, `Nat.succ (Nat.succ Nat.zero)`, etcétera, y nada más. De este “y nada más” Lean extrae automáticamente dos herramientas:

- Un *principio de recursión*: para definir una función sobre un tipo inductivo basta decir qué hace con cada constructor. Lo usaremos, por ejemplo, para definir la relación de satisfacción por recursión sobre la estructura de las fórmulas.
- Un *principio de inducción*: para demostrar que todos los términos del tipo cumplen una propiedad basta demostrarla para cada constructor, suponiéndola para sus argumentos (hipótesis de inducción). La inducción usual sobre $\mathbb{N}$ es el caso particular para `Nat`; nosotros usaremos, además, la inducción estructural sobre fórmulas.

Hay un uso de los tipos inductivos menos evidente y central en este trabajo: se pueden definir proposiciones inductivas, es decir, familias de proposiciones indexadas, términos de tipo $A \rightarrow \text{Prop}$, cuyos constructores generan las demostraciones de cada miembro de la familia. Nuestra noción de demostrabilidad ($\vdash \varphi$) será una proposición inductiva cuyos constructores son, literalmente, los axiomas y reglas del sistema **K**. Una demostración en **K** queda así representada como un árbol finito de aplicaciones de constructores. La consecuencia es muy potente, pues el principio de inducción asociado permite hacer inducción sobre las demostraciones, que es exactamente la técnica con la que se prueba el teorema de corrección (“si los axiomas son válidos y las reglas preservan la validez, todo teorema es válido”).

Por último, las estructuras (`structure`) se apoyan en los tipos inductivos. Lean traduce cada estructura en un tipo inductivo con un único constructor y genera automáticamente las proyecciones que dan acceso a los campos. Sirven para empaquetar varios datos en un solo objeto, como un marco de Kripke (un tipo de mundos y una relación entre ellos) o un modelo (un marco y una valoración):

```
1 structure KripkeFrame where
2   World : Type
3   R : World → World → Prop
```

```

4
5 structure KripkeModel (VP : Type) extends KripkeFrame where
6   V : World → VP → Prop

```

Si $F : \text{KripkeFrame}$, se accede a sus campos con la notación $F.\text{World}$ y $F.R$. La palabra `extends` indica que una estructura hereda los campos de otra: un `KripkeModel` tiene los dos campos de `KripkeFrame` además del suyo propio, de modo que si $M : \text{KripkeModel VP}$, tanto $M.R$ como $M.V$ son legítimas. Estas dos declaraciones son exactamente las que se usarán en el capítulo 4 para definir la semántica.

2.3. Definiciones y enunciados

Con el fundamento anterior, escribir Lean consiste en declarar términos y sus tipos. Las declaraciones que usamos son cuatro:

- `def` introduce una definición con nombre. Por ejemplo,

```

1 def EsPar (n : Nat) : Prop := ∃ k : Nat, n = 2 * k

```

define la propiedad de ser par: recibe un natural n y devuelve la proposición que afirma que n es el doble de algún k . Entre el nombre y los dos puntos van los argumentos con su tipo; lo que sigue a los dos puntos es el tipo del resultado (aquí `Prop`, porque lo definido es un enunciado y no un número), y lo que sigue a `:=` es la definición. Así se definirán propiedades en el capítulo 4 con nociones como la validez o la consistencia.

- `theorem` declara un teorema: su “tipo” es el enunciado (una proposición) y su “definición” es la demostración (un término de ese tipo). Por ejemplo, `theorem cuatro_es_par : EsPar 4 := ⟨2, rfl⟩` demuestra que 4 es par aportando el testigo $k = 2$ y la comprobación de que $4 = 2 * 2$. Compárese con la declaración de `EsPar`: la forma es idéntica, nombre, tipo y término tras `:=`; lo único que cambia es que ahora el tipo es un enunciado. Que teoremas y definiciones tengan la misma forma es un ejemplo más de correspondencia de Curry–Howard.
- `example` es un teorema sin nombre: se verifica y se olvida. La declaración anterior escrita como `example : EsPar 4 := ⟨2, rfl⟩` comprueba exactamente lo mismo, pero no deja nada utilizable después. Lo usamos para ejemplos o verificaciones que no se necesitan más adelante.

- `variable` declara parámetros comunes a varias declaraciones. En nuestro código, por ejemplo, `variable {VP : Type}` fija de una vez el tipo de las variables proposicionales para todo el desarrollo.

Argumentos implícitos

En algunas declaraciones aparecen argumentos entre llaves en lugar de entre paréntesis. Las llaves indican que el argumento es implícito: quien usa la definición no lo escribe, y Lean lo infiere del contexto. Por ejemplo, un lema sobre listas que afirme que invertir dos veces devuelve la lista de partida,

```
1 theorem rev_rev {α : Type} (l : List α) : (l.reverse).reverse = l :=
2   List.reverse_reverse l
```

tiene el tipo α implícito, de modo que basta escribir `rev_rev [1, 2, 3]` y Lean deduce que α es `Nat` sin más que mirar la lista. Si α se hubiera declarado explícito, `(α : Type)`, habría que escribir `rev_rev Nat [1, 2, 3]`, repitiendo una información que ya está contenida en el segundo argumento. Cuando interesa dar un argumento implícito a mano, se antepone `@` al nombre del resultado, lo que vuelve explícitos todos sus argumentos: `@rev_rev Nat [1, 2, 3]`.

En el código se ha seguido la convención habitual en Lean: marcar como implícito todo argumento que pueda deducirse del tipo de los demás, y dejar explícitos los que no. En el capítulo 4, el tipo de variables proposicionales sobre el que se construye todo el desarrollo será un argumento implícito de casi todas las definiciones y teoremas. Allí reaparecerá también la notación `@`, en patrones como `| @mp χ ξ _ _ ihχξ ihχ =>`, que nombran los argumentos implícitos de un constructor al razonar por inducción sobre las derivaciones.

Definiciones por recursión

Como anticipamos, una función sobre un tipo inductivo puede definirse por casos sobre los constructores, mediante ajuste de patrones (*pattern matching*):

```
1 def suma : Nat → Nat → Nat
2   | Nat.zero,   n => n
3   | Nat.succ m, n => Nat.succ (suma m n)
```

Cada línea `| patrón => valor` cubre un constructor; Lean comprueba que los casos son exhaustivos y que las llamadas recursivas se hacen sobre subtérminos estrictos del argumento, esto es, sobre las piezas con las que el constructor lo formó. En `suma`, el patrón

`Nat.succ m` da lugar a una llamada sobre `m`. Como los términos de un tipo inductivo son árboles finitos, esta relación de subtérmino está bien fundada y garantiza que la definición termina².

Sobre estas definiciones se apoya la noción de *igualdad definicional*: dos expresiones son definicionalmente iguales si una se obtiene de la otra desplegando definiciones y computando. Por ejemplo, `suma Nat.zero n` es definicionalmente `n`, sin más que aplicar la primera cláusula. Las igualdades definicionales no necesitan demostración, pues la táctica `rf1` (sección 2.5) las cierra de inmediato, y en la formalización señalaremos varios lemas cuya demostración completa es, precisamente, `rf1`. La relación de satisfacción del capítulo 4 se define exactamente así, con una cláusula por cada conectiva del lenguaje modal.

Notación

Lean permite introducir notación infija y prefija con niveles de precedencia. Por ejemplo,

```
1 local infixr:60 " → " => ModalFormula.imp
2 local prefix:70 "□"   => ModalFormula.box
```

declara que $\varphi \rightarrow \psi$ abrevia `ModalFormula.imp  $\varphi$   $\psi$` y que $\Box\varphi$ se abrevia como `ModalFormula.box  $\varphi$` . El número tras los dos puntos es la precedencia. A mayor número, más fuerte liga. Así, $\Box\varphi \rightarrow \psi$ se lee $(\Box\varphi) \rightarrow \psi$. La `r` de `infixr` indica asociatividad a la derecha (la convención habitual para la implicación) y el modificador `local` restringe la notación al fichero actual. Como se discute en la sección 4.1, elegimos deliberadamente símbolos distintos de los de Lean ($\rightarrow$ frente a $\rightarrow$, $\sim$ frente a $\neg$, $\wedge$ frente a $\wedge \dots$) para que siempre sea visible si una expresión es una fórmula modal o una proposición de Lean.

2.4. El modo táctico y el InfoView

Demostrar un teorema es escribir un término de su tipo, y para enunciados sencillos puede hacerse directamente. Por ejemplo, el *modus ponens*:

```
1 theorem mp {p q : Prop} (h : p → q) (hp : p) : q := h hp
```

La demostración es la aplicación `h hp`, tal como vimos en la sección 2.2: la implicación es una función y aplicarla es razonar. Pero construir a mano el término de una demostración larga es impracticable. Para eso existe el *modo táctico*: escribiendo `by`

²Lean admite además recursión bien fundada respecto de una medida arbitraria, declarada con `termination_by`, para las definiciones en las que el criterio estructural no basta. En este trabajo no se ha necesitado en ningún momento.

tras el enunciado, en lugar de un término se da una sucesión de instrucciones que van construyendo el término por nosotros.

El modo táctico se entiende mejor con la noción de estado de la demostración, que el InfoView muestra en todo momento. Un estado consta de uno o varios objetivos (*goals*); cada objetivo tiene un contexto de hipótesis con nombre ($n : \text{Nat}, h : p, \dots$) y una meta, la proposición que falta demostrar, señalada con el símbolo $\vdash$. Cada táctica transforma el estado: puede simplificar la meta, añadir hipótesis, dividir el objetivo en varios o cerrarlo. La demostración acaba cuando no quedan objetivos y el InfoView muestra el mensaje “*Goals accomplished!*”. La figura 2.1 muestra el aspecto de esta ventana durante una demostración.

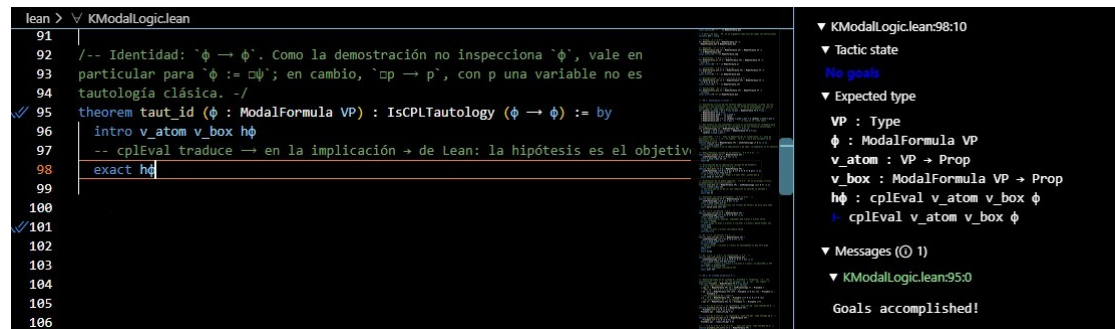


Figura 2.1: El InfoView de Lean durante una demostración en modo táctico. A la izquierda, el editor con el cursor situado tras una táctica; a la derecha, el estado de la demostración en ese punto.

Veámoslo con un ejemplo elemental: si p es cierta, entonces p se sigue de cualquier otra cosa. En los comentarios se indica el estado de la demostración antes y después de cada táctica, tal como lo mostraría el InfoView.

```

1  example (p q : Prop) : p → (q → p) := by
2    -- ⊢ p → q → p
3    intro hp
4    -- hp : p
5    -- ⊢ q → p
6    intro hq
7    -- hp : p,   hq : q
8    -- ⊢ p
9    exact hp
10   -- Goals accomplished!

```

La meta inicial es una implicación anidada. La táctica `intro hp` supone el antecedente, es el “supongamos p ” de una demostración informal, y lo añade al contexto con el nombre

`hp`, dejando como meta el consecuente. `intro hq` hace lo propio con la segunda implicación. La meta es entonces p , que es exactamente la hipótesis `hp`, y `exact hp` la señala y cierra el objetivo. Estas dos tácticas, junto con el resto de las empleadas en el trabajo, se detallan en la sección 2.5.

Merece la pena subrayar la relación entre los dos modos: las tácticas no son una lógica aparte, sino un lenguaje de órdenes para construir el mismo término que podríamos escribir a mano. En el ejemplo, el término construido es `fun hp hq => hp`. A lo largo del trabajo combinamos ambos estilos según cuál resulte más legible, alguna demostración corta se escribe directamente como término, igual que el *modus ponens* de arriba, con preferencia por el modo táctico con las hipótesis intermedias explícitas (`have`), que es el que más se parece a una demostración escrita en prosa.

2.5. Catálogo de tácticas empleadas

Esta sección recoge todas las tácticas que aparecen en la formalización, agrupadas por función, con su significado matemático y una referencia a algún lugar del capítulo 4 donde se utilicen. La tabla 2.1 sirve de resumen y de índice.

Táctica	Lectura matemática	Uso destacado
<code>intro</code>	sea... / supongamos...	en casi todos los lemas
<code>exact</code>	que es exactamente...	en casi todos los lemas
<code>apply</code>	basta ver... (razonar hacia atrás)	lema de verdad (§4.6.5)
<code>have</code>	afirmación intermedia	regla RE (§4.3)
<code>rfl</code>	por definición	<code>satisfies_neg</code> (§4.4)
<code>constructor</code>	separar $\leftrightarrow$, $\wedge$ en partes	<code>satisfies_dia</code> (§4.4)
<code>rintro</code> , <code>rcases</code> , <code>obtain</code>	descomponer hipótesis	Lindenbaum (§4.6.3)
<code>refine</code>	esquema con huecos	lema de existencia (§4.6.5)
<code>by_contra</code>	reducción al absurdo	varios, completitud (§4.6.6)
<code>by_cases</code>	distinción de casos clásica	propiedades de los MCS (§4.6.2)
<code>contradiction</code>	hipótesis contradictorias	tautologías del $\leftrightarrow$ (§4.2)
<code>induction ... with</code>	inducción	corrección (§4.5)
<code>subst</code>	sustituir usando una igualdad	teorema de deducción (§4.6.1)
<code>simp</code>	simplificación automática	un único uso (§4.6.1)

Tabla 2.1: Resumen de las tácticas empleadas en la formalización.

2.5.1. Tácticas básicas de manipulación de objetivos

intro. Si la meta es una implicación $p \rightarrow q$ o un universal $\forall x, P x$, la táctica `intro h` mueve el antecedente al contexto como nueva hipótesis `h` (o introduce un `x` genérico) y deja como meta el consecuente. Corresponde al “supongamos que” y al “sea x arbitrario”

del lenguaje matemático. Admite varios nombres seguidos:

```
1 --  $\vdash p \rightarrow q \rightarrow p$ 
2 intro hp hq
3 --  $hp : p, \quad hq : q$ 
4 --  $\vdash p$ 
```

Es, con diferencia, la táctica más usada del trabajo, ya que casi todas nuestras nociones (tautología, validez, consistencia) son cadenas de cuantificadores e implicaciones.

exact. Cierra la meta proporcionando un término que la demuestra, construido con las hipótesis disponibles. En el caso más simple el término es una hipótesis: con $hp : p$ y meta $\vdash p$, basta **exact** hp . Pero puede ser cualquier término del tipo adecuado:

```
1 --  $h : p \rightarrow q, \quad hp : p$ 
2 --  $\vdash q$ 
3 exact h hp
```

La implicación se aplica a la hipótesis y el término resultante cierra la meta. En el trabajo aparece a menudo en esta segunda forma, encadenando varias aplicaciones en un solo término.

apply. Razona hacia atrás: reduce la meta al antecedente de una implicación ya disponible (“para ver q , por h basta ver p ”).

```
1 --  $h : p \rightarrow q$ 
2 --  $\vdash q$ 
3 apply h
4 --  $\vdash p$ 
```

Es la contrapartida de **exact**: en lugar de dar el término completo de una vez, se avanza dejando pendiente lo que falta. La usamos, entre otros sitios, en el caso de la implicación del lema de verdad (sección 4.6).

have. Introduce una afirmación intermedia junto con su propia demostración para poder usar como hipótesis: **have** $h : p := \text{término}$ o bien **have** $h : p := \text{by } \dots$. Tras ella, h queda disponible en el contexto como cualquier otra hipótesis.

```
1 --  $h : p \rightarrow q, \quad hp : p$ 
2 --  $\vdash r$ 
3 have hq :  $q := h \ hp$ 
```

```

4 -- h : p → q,  hp : p,  hq : q
5 -- ⊢ r

```

Es la táctica que estructura las demostraciones largas como sucesiones de pasos con nombre (“primero observamos... , de donde...”). La demostración de la regla RE en la sección 4.3 es el ejemplo más evidente: siete `have` encadenados que se leen exactamente igual que la demostración en papel.

rfl. Cierra metas de la forma $a = b$ o $a \leftrightarrow b$ cuando ambos lados son definicionalmente iguales (sección 2.3), lo que se comprueba calculando:

```

1 -- ⊢ 2 + 2 = 4
2 rfl
3 -- Goals accomplished!

```

Que un lema se demuestre con `rfl` aporta además información importante: significa que no hay contenido matemático nuevo, solo se despliegan definiciones. Lo veremos en `satisfies_neg` y en varios casos de la inducción de `cplEval_iff_satisfies`, en la sección 4.4.

2.5.2. Construir y descomponer estructuras lógicas

constructor. Cuando la meta es una conjunción o un bicondicional, la divide en sus componentes, generando un objetivo por cada una (ante $\vdash p \leftrightarrow q$, las dos implicaciones):

```

1 -- ⊢ p ∧ q
2 constructor
3 -- ⊢ p
4 -- ⊢ q

```

El InfoView los muestra como objetivos separados, para delimitar la demostración de cada uno. Alternativamente puede darse el término par directamente con el constructor anónimo (sección 2.2). La demostración final del teorema de adecuación es, simplemente, `<soundness, completeness>`.

rcases, rintro, obtain. Son las tácticas duales de `constructor`: en lugar de dividir la meta, descomponen hipótesis compuestas mediante patrones.

```

1 -- h : ∃ x, P x ∧ Q x
2 rcases h with ⟨x, hP, hQ⟩
3 -- x : A,  hP : P x,  hQ : Q x

```

La táctica sustituye h por el testigo x y las dos componentes de la conjunción, y los patrones pueden anidarse tanto como haga falta. Con $h : p \vee q$, en cambio, `rcases h with hp | hq` genera dos objetivos, uno por cada caso: la barra separa alternativas y los corchetes angulares agrupan componentes simultáneas. La táctica `rintro` combina `intro` con esta descomposición en un solo paso, y `obtain` hace lo propio con el resultado de aplicar un teorema: `obtain ⟨x, hP, hQ⟩ := teorema hp` aplica el teorema y descompone lo que devuelve. La lectura matemática es siempre la misma: “sea x el objeto que nos proporciona el teorema, con tales propiedades”. En el trabajo aparecen sobre todo al usar el lema de Lindenbaum y el lema de existencia (sección 4.6).

refine. Es un `exact` con huecos: se escribe el esqueleto del término y se dejan `?_` en las partes pendientes, que se convierten en nuevos objetivos.

```
1 -- hp : p
2 -- ⊢ p ∧ q
3 refine ⟨hp, ?_⟩
4 -- ⊢ q
```

Resulta ideal cuando la parte creativa, elegir el testigo de un existencial, es previa a las comprobaciones rutinarias, que es exactamente la situación del lema de existencia de la sección 4.6.

2.5.3. Razonamiento clásico

by_contra. Reducción al absurdo: para demostrar p , la táctica `by_contra h` añade la hipótesis $h : \neg p$ y cambia la meta a `False`, de modo que basta llegar a una contradicción.

```
1 -- ⊢ p
2 by_contra h
3 -- h : ¬p
4 -- ⊢ False
```

Es un principio genuinamente clásico que equivale a la eliminación de la doble negación. Por ejemplo, lo vemos en la demostración de completitud (sección 4.6), y aparece puntualmente también cada vez que hay que pasar de una negación a una afirmación, como en la inferencia $\neg\forall x \neg P(x) \Rightarrow \exists x P(x)$, que no es demostrable intuicionistamente. Volvemos sobre esta cuestión en la sección 2.6.

by_cases. Distinción de casos sobre una proposición cualquiera, abriendo un objetivo por cada posibilidad:

```

1 --  $\vdash q$ 
2 by_cases h : p
3 -- caso 1:  $h : p, \vdash q$ 
4 -- caso 2:  $h : \neg p, \vdash q$ 

```

Es la herramienta natural para razonar por casos sin disponer de una disyunción previa: en el trabajo la usamos para las propiedades de los conjuntos maximales consistentes (sección 4.6), del tipo “o bien Γ deduce φ , o bien no lo hace”.

contradiction. Cierra el objetivo cuando el contexto contiene una contradicción explícita, como $h : p$ junto con $h' : \neg p$ o una hipótesis $h : \text{False}$. En tal caso la meta, sea cual sea, queda demostrada:

```

1 --  $h : p, h' : \neg p$ 
2 --  $\vdash q$ 
3 contradiction
4 -- Goals accomplished!

```

Relacionadas con lo anterior están dos funciones (no tácticas) que usamos para rematar razonamientos: `False.elim h`, que a partir de $h : \text{False}$ demuestra cualquier cosa (*ex falso quodlibet*), y `absurd h h'`, que hace lo mismo a partir de $h : p$ y $h' : \neg p$.

2.5.4. Inducción y reescritura

induction ... with. Aplica el principio de inducción del tipo inductivo correspondiente, abriendo un objetivo por constructor; la sintaxis `with | caso1 ... | caso2 ...` permite nombrar en cada caso los argumentos y las hipótesis de inducción. Un ejemplo con listas:

```

1 def longitud {α : Type} : List α → Nat
2 | []      => 0
3 | _ :: xs => longitud xs + 1
4
5 example {α β : Type} (f : α → β) (l : List α) :
6   longitud (l.map f) = longitud l := by
7   induction l with
8   | nil      => rfl
9   | cons x xs ih => simp [longitud, ih]

```

El tipo `List α` tiene dos constructores: la lista vacía `[]` y la que antepone un elemento a otra lista, `x :: xs`. La inducción abre por tanto dos objetivos: el caso `nil`, que es la base y aquí se cierra de forma inmediata con `rfl`; y el caso `cons`, que recibe el elemento `x`, la cola `xs` y la hipótesis de inducción `ih`, que afirma la propiedad para `xs`. Lo mismo ocurre con cualquier otro tipo inductivo: hay un caso por constructor, y los constructores recursivos vienen acompañados de su hipótesis de inducción.

En este trabajo la inducción se aplica sobre dos clases de objetos bien distintas, y es oportuno separarlas desde el principio:

- *Inducción estructural sobre fórmulas* (tipo `ModalFormula`): los casos son los cuatro constructores, átomo, $\perp$, implicación y caja, y los dos últimos traen hipótesis de inducción sobre las subfórmulas. Así se demuestran el lema puente de la corrección y el lema de verdad (secciones 4.5 y 4.6).
- *Inducción sobre derivaciones* (tipos `Provable` y `DeducibleFrom`): como una demostración formal es también un objeto inductivo, se puede razonar “por inducción sobre la demostración”, con un caso por axioma o regla. Usada para la corrección, el teorema de deducción y la monotonía de la deducibilidad.

induction ... generalizing. Cuando la propiedad que se demuestra por inducción depende de otra variable que cambia dentro de los casos, esa variable debe generalizarse antes de hacer la inducción, o las hipótesis quedarán fijadas a su valor actual y no servirán. Escribir `induction x generalizing` y lo consigue. Es el análogo formal de la práctica habitual en papel de enunciar la hipótesis de inducción “para todo y ”. En el lema de verdad resulta imprescindible, y allí explicamos por qué (sección 4.6).

subst. Dada una hipótesis de igualdad `heq : x = a`, elimina `x` sustituyéndolo por `a` en la meta y en las demás hipótesis, y descarta la igualdad.

```

1 -- heq : x = a,  h : P x
2 -- ⊢ Q x
3 subst heq
4 -- h : P a
5 -- ⊢ Q a

```

Es lo que en papel se hace al decir “pero entonces x es a ”. Aparece una única vez, en el teorema de deducción: al descomponer la pertenencia de una fórmula a `insert φ Γ`, uno de los dos casos deja como hipótesis que dicha fórmula es φ , y `subst` la elimina (subsección 4.6.1).

simp. Es la principal táctica de automatización de Lean: reescribe la meta (o una hipótesis, con `simp at h`) usando una base de ecuaciones marcadas como reglas de simplificación, y cierra el objetivo si llega a algo trivial. Es muy cómoda pero tiene un precio: oculta los pasos que da, de modo que su uso indiscriminado produce demostraciones cortas pero ilegibles. Por eso en este trabajo aparece una única vez, para descartar una hipótesis imposible (subsección 4.6.1):

```
1 -- h :  $\varphi \in (\emptyset : \text{Set } (\text{ModalFormula } VP))$ 
2 simp at h
3 -- Goals accomplished!
```

La hipótesis afirma que una fórmula pertenece al conjunto vacío; `simp` la reduce a `False` y cierra el objetivo.

2.6. Mathlib, la lógica clásica y el lema de Zorn

Nuestro fichero comienza con dos importaciones:

```
1 import Mathlib.Tactic
2 import Mathlib.Order.Zorn
```

La primera carga el modo táctico extendido de Mathlib; la segunda, el lema de Zorn. De Mathlib usamos tres cosas, que repasamos a continuación.

Conjuntos

El tipo `Set  $\alpha$` representa los subconjuntos de un tipo α ; técnicamente, un conjunto es su función característica, `Set  $\alpha := \alpha \rightarrow \text{Prop}$` , y $x \in A$ es la proposición $A\ x$. Sobre él están definidos $\subseteq$, $\cup$, $\emptyset$, la inserción de un elemento `insert a A` (que usamos constantemente para “añadir una fórmula a un conjunto de hipótesis”) y la unión de una familia, $\bigcup_0 C$ (notación para `Set.sUnion C`, la unión de todos los conjuntos que pertenecen a la colección `C`). De la biblioteca tomamos lemas elementales de manipulación cuyo nombre describe su contenido: `Set.mem_insert` ($a \in \text{insert } a\ A$), `Set.mem_insert_iff` ($x \in \text{insert } a\ A \leftrightarrow x = a \vee x \in A$), `Set.subset_insert`, `Set.mem_insert_of_mem`, `Set.mem_sUnion` y `Set.subset_sUnion_of_mem`. Aprender a encontrar estos lemas (por su nomenclatura sistemática, con la búsqueda del editor o con herramientas como `exact?`) ha sido parte del aprendizaje de Lean.

Lógica clásica

El núcleo de Lean es constructivo: por defecto no se dispone del principio del tercero excluido. Mathlib, como la práctica matemática habitual, trabaja en lógica clásica, axiomatizada mediante `Classical.choice`; de ahí derivan el tercero excluido y las tácticas `by_contra` y `by_cases` que acabamos de describir. Nuestro desarrollo es clásico, igual que el texto de referencia [4]. La propia semántica evalúa cada fórmula a verdadera o falsa, la eliminación de la doble negación es una de nuestras tautologías básicas, y la completitud usa el lema de Zorn, equivalente al axioma de elección. No es una limitación de Lean, sino una elección, puesto que sí existe una lógica modal intuicionista, pero no es el objeto de este trabajo.

El lema de Zorn

Del orden por inclusión entre conjuntos, Mathlib ofrece la siguiente versión del lema de Zorn, que es la que aplicaremos en el lema de Lindenbaum (sección 4.6):

```
1 theorem zorn_subset_nonempty (S : Set (Set  $\alpha$ ))
2   (H :  $\forall c \subseteq S, \text{IsChain } (\cdot \subseteq \cdot) c \rightarrow c.\text{Nonempty} \rightarrow$ 
3      $\exists ub \in S, \forall s \in c, s \subseteq ub$ )
4   (x) (hx :  $x \in S$ ) :
5    $\exists m, x \subseteq m \wedge \text{Maximal } (\cdot \in S) m$ 
```

Leído con calma: S es una familia de conjuntos. La hipótesis H pide que toda cadena no vacía c contenida en S tenga una cota superior dentro de S^3 . La conclusión da, para cada $x \in S$, un elemento m de la familia con $x \subseteq m$ que es maximal en ella.

El argumento x se declara sin indicar su tipo, y Lean lo deduce de la hipótesis $hx : x \in S$, que obliga a que sea un `Set  $\alpha$` . A su vez, la maximalidad se expresa con el predicado `Maximal  $(\cdot \in S) m$` , que significa que m cumple la propiedad de pertenecer a S y que ningún elemento que la cumpla lo contiene estrictamente. De él extraeremos las dos piezas que necesitamos con `.1` y `.2`, la pertenencia y la maximalidad propiamente dicha.

En nuestro caso S será la familia de los conjuntos consistentes de fórmulas y la cota superior de una cadena, su unión. El trabajo matemático consistirá en demostrar que esa unión sigue siendo consistente.

³Que c sea una cadena, esto es, una subfamilia totalmente ordenada por $\subseteq$, es lo que expresa `IsChain  $(\cdot \subseteq \cdot) c$` . La notación `$(\cdot \subseteq \cdot)$` es una función anónima abreviada: el punto medio marca los huecos de los argumentos, de modo que equivale a `fun a b => a  $\subseteq$  b`.

Capítulo 3

La lógica modal $\mathbf{K}$

Este capítulo contiene la parte matemática del trabajo en su versión “de papel”: la sintaxis del lenguaje modal básico, el sistema axiomático $\mathbf{K}$, la semántica de Kripke y los enunciados de corrección y completitud, junto con el mapa de la demostración de esta última. Todo ello sigue en lo esencial a Blackburn, de Rijke y Venema [4], con las adaptaciones, que iremos señalando, que se realizan pensando en la formalización del capítulo 4. Cada resultado que después se formaliza lleva en su encabezado, en letra de máquina de escribir, el nombre con el que aparece en el código, de modo que pueda localizarse sin dificultad en el capítulo 4.

3.1. ¿Necesidad y posibilidad?

La lógica proposicional clásica trata cada enunciado como verdadero o falso, sin matices. Pero el lenguaje natural distingue *modos* de verdad. No es lo mismo “llueve” que “necesariamente llueve” o que “es posible que llueva”. La lógica modal introduce dos operadores para capturar esa distinción: $\Box\varphi$ (“ φ es necesaria”) y $\Diamond\varphi$ (“ φ es posible”). Ambos están ligados por una dualidad esperable. Algo es posible exactamente cuando su negación no es necesaria, $\Diamond\varphi \equiv \neg\Box\neg\varphi$. En nuestro ejemplo de lluvia, “es posible que llueva” es lo mismo que “no es necesario que no llueva”.

Parte del atractivo de la lógica modal es que la lectura de los operadores no está fijada. Según la aplicación, un mismo par $\Box, \Diamond$ admite lecturas muy distintas:

En todos los casos la dualidad se mantiene: lo permitido es lo que no está prohibido, lo compatible con el conocimiento es lo que no se sabe falso, lo que ocurrirá alguna vez es lo que no siempre será falso. Cada lectura sugiere, en cambio, principios distintos. Por ejemplo, “lo sabido es cierto” ($\Box\varphi \rightarrow \varphi$) es razonable para el conocimiento, y también para el tiempo si se admite el instante presente, pero no para la obligación (que algo sea obligatorio no lo hace cierto). Esta multiplicidad explica que exista toda una familia de

Lectura	$\Box\varphi$	$\Diamond\varphi$
Alética (la clásica)	φ es necesaria	φ es posible
Epistémica	φ se sabe	φ no se sabe falsa
Temporal	φ será siempre cierta	φ será cierta alguna vez
Deónica	φ es obligatoria	φ está permitida

Tabla 3.1: Cuatro lecturas habituales del par $\Box, \Diamond$. En la temporal se cuantifica sobre los instantes futuros; en la epistémica, $\Diamond\varphi$ equivale a “ φ es compatible con lo que se sabe” y los operadores suelen escribirse $K\varphi$ y $\widehat{K}\varphi$.

lógicas modales.

La que estudiamos aquí, la lógica **K** (por Kripke), es la base común y más débil de todas ellas. No compromete ninguna lectura y sirve de punto de partida para las demás, que se obtienen añadiendo nuevos axiomas. La lectura epistémica reaparecerá en el capítulo 5.

3.2. Sintaxis

Definición 3.1 (Lenguaje modal básico). Fijado un conjunto **VP** de *variables proposicionales*, el conjunto **Form** de *fórmulas modales* se define por la siguiente gramática:

$$\varphi ::= p \mid \perp \mid \varphi \rightarrow \varphi \mid \Box\varphi, \quad p \in \mathbf{VP}.$$

Es decir, **Form** es el menor conjunto que contiene las variables y la constante $\perp$ (“falso”) y está cerrado bajo la implicación y bajo el operador $\Box$.

Fijamos también la convención de nombres que seguirá todo el trabajo: las letras latinas minúsculas p, q, r denotan siempre variables proposicionales, es decir, elementos de **VP**, mientras que las griegas minúsculas φ, ψ, χ (y ocasionalmente α, β) denotan fórmulas cualesquiera. La distinción no es cosmética: $\Box p \rightarrow p$ es una fórmula concreta, mientras que $\Box\varphi \rightarrow \varphi$ es un esquema, con una instancia por cada fórmula φ , y veremos en la sección 3.3 que hay propiedades de la primera que no comparten todas las instancias del segundo.

Adoptamos deliberadamente un lenguaje primitivo mínimo, formado solo por $\perp, \rightarrow$ y $\Box$. El resto de conectivas se introducen como abreviaturas:

$$\begin{aligned} \top &:= \perp \rightarrow \perp, & \neg\varphi &:= \varphi \rightarrow \perp, \\ \varphi \vee \psi &:= \neg\varphi \rightarrow \psi, & \varphi \wedge \psi &:= \neg(\varphi \rightarrow \neg\psi), \\ \varphi \leftrightarrow \psi &:= (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi), & \Diamond\varphi &:= \neg\Box\neg\varphi. \end{aligned}$$

Las definiciones de $\vee$ y $\wedge$ son las clásicas. La fórmula $\varphi \vee \psi$ dice “si no φ , entonces ψ ”, y $\varphi \wedge \psi$ niega que φ implique la negación de ψ . La ventaja de trabajar con un lenguaje mínimo es que las definiciones y demostraciones por inducción estructural tienen solo cuatro casos. Además, en la formalización, cada constructor del tipo de fórmulas es un caso en cada demostración, y menos primitivas significa menos casos que demostrar. Las conectivas derivadas reciben además su significado “gratis”, como comprobaremos en la sección 4.4.

Esta es la primera de las adaptaciones. Blackburn, de Rijke y Venema [4, Def. 1.9] toman como primitivos $\perp$, $\neg$, $\vee$ y $\Diamond$, y es $\Box$ el que se define, como $\Box\varphi := \neg\Diamond\neg\varphi$; la implicación es allí la abreviatura $\varphi \rightarrow \psi := \neg\varphi \vee \psi$. Nosotros invertimos la elección. Ambos lenguajes son intertraducibles (cada fórmula de uno se reescribe en el otro sin alterar su significado respecto a la semántica de la sección 3.4), y los propios autores reconocen que su preferencia por $\Diamond$ obedece a la comodidad para el trabajo algebraico de capítulos posteriores del libro. Pensando en la formalización, $\Box$ es la opción natural, porque su cláusula semántica es una cuantificación universal, más cómoda de manejar en un asistente de demostración que la existencial de $\Diamond$ (sección 4.4). La misma elección hace innecesario uno de los axiomas del sistema de Blackburn, como veremos en la sección 3.3.

Obsérvese también que no imponemos ninguna condición sobre el conjunto $\mathbf{VP}$. Blackburn [4, p. 10] lo supone habitualmente numerable, hipótesis que interviene en su demostración del lema de Lindenbaum; nosotros podremos prescindir de ella (sección 3.5, paso 3).

3.3. El sistema axiomático **K**

Un sistema axiomático determina qué fórmulas son demostrables a partir de unos axiomas mediante unas reglas de inferencia. El sistema **K** toma como punto de partida todo el cálculo proposicional clásico y le añade lo mínimo para razonar con $\Box$. A las fórmulas bajo el operador $\Box$ las llamaremos fórmulas *encajonadas* (del inglés *boxed*, [4, pp. 19 y 34]). Precisamos primero ese punto de partida.

Definición 3.2 (Tautología clásica). Una *valoración booleana* es un par de funciones $v = (v_a, v_\Box)$, con $v_a: \mathbf{VP} \rightarrow \{0, 1\}$ y $v_\Box: \mathbf{Form} \rightarrow \{0, 1\}$, que asignan valores de verdad (1 verdadero, 0 falso) a las variables y a las fórmulas encajonadas: $v_a(p)$ es el valor de la variable p , y $v_\Box(\psi)$, el de la fórmula $\Box\psi$. Fijada una valoración, toda fórmula recibe un

valor $\hat{v}(\varphi)$ por recursión:

$$\begin{aligned}\hat{v}(p) &= v_a(p), \\ \hat{v}(\perp) &= 0, \\ \hat{v}(\varphi \rightarrow \psi) &= \begin{cases} 0 & \text{si } \hat{v}(\varphi) = 1 \text{ y } \hat{v}(\psi) = 0, \\ 1 & \text{en otro caso,} \end{cases} \\ \hat{v}(\Box\psi) &= v_{\Box}(\psi).\end{aligned}$$

Una fórmula φ es una *tautología clásica* si $\hat{v}(\varphi) = 1$ para toda valoración booleana v .

Esta manera de evaluar significa que, a efectos proposicionales, cada subfórmula $\Box\psi$ es un *átomo opaco*. Átomo, porque la valoración le asigna un valor de verdad directamente, igual que a una variable, en lugar de descomponerla; opaco, porque la última cláusula no evalúa ψ , de modo que el valor de $\Box\psi$ es independiente de lo que haya dentro de la caja. Por ejemplo, $\Box\Box p \rightarrow \Box p$ no es una tautología clásica: los átomos opacos $\Box\Box p$ y $\Box p$ son fórmulas distintas, y cualquier valoración con $v_{\Box}(\Box p) = 1$ y $v_{\Box}(p) = 0$ la hace falsa. La definición 3.2 coincide con la formalización desarrollada en el Capítulo 4. v_a y $v_{\Box}$ son los argumentos `v_atom` y `v_box` de la función `cplEval` del código, que computa $\hat{v}$.

Debemos contrastar dos casos. La fórmula $\Box\psi \rightarrow \Box\psi$ es una tautología clásica para cualquier ψ , pues su átomo opaco recibe el mismo valor a los dos lados de la implicación; en cambio $\Box p \rightarrow p$ no lo es, porque cualquier valoración con $v_{\Box}(p) = 1$ y $v_a(p) = 0$ la hace falsa. Aquí es esencial la convención de la sección 3.2: $\Box p \rightarrow p$ es una fórmula concreta, con p una variable, mientras que el esquema $\Box\varphi \rightarrow \varphi$ sí tiene instancias que son tautologías clásicas, como $\Box\top \rightarrow \top$, verdadera bajo toda valoración porque ninguna hace falsa $\top$.

La formulación habitual en la literatura es otra. Recordemos que una *tautología proposicional* es una fórmula del fragmento sin $\Box$ de nuestro lenguaje, construida solo con variables, $\perp$ y $\rightarrow$, verdadera bajo toda asignación de valores de verdad a sus variables, y que una *instancia* suya se obtiene reemplazando uniformemente cada variable por una fórmula modal cualquiera. Blackburn admite como axiomas precisamente esas instancias [4, Def. 1.39], sin definir la noción por separado. Le basta con que la fórmula tenga la forma de una tautología proposicional. La definición 3.2 selecciona las mismas fórmulas, como comprobaremos más abajo, pero por un camino distinto. Construimos esta formulación porque la versión por sustitución exigiría introducir ese lenguaje auxiliar, una operación de sustitución y un lema que las conecte, mientras que evaluar con dos asignaciones es una recursión inmediata sobre fórmulas.

Definición 3.3 (El sistema **K**). La relación de demostrabilidad $\vdash \varphi$ (“ φ es un teorema de **K**”) es la menor relación que cumple:

(CPL) $\vdash \varphi$ para toda *tautología clásica* φ (definición 3.2);

(K) $\vdash \Box(\varphi \rightarrow \psi) \rightarrow (\Box\varphi \rightarrow \Box\psi)$, para cualesquiera fórmulas φ, ψ (*axioma de distribución*);

(MP) si $\vdash \varphi \rightarrow \psi$ y $\vdash \varphi$, entonces $\vdash \psi$ (*modus ponens*);

(N) si $\vdash \varphi$, entonces $\vdash \Box\varphi$ (*regla de necesidad*).

Observación 3.4. Detengámonos en (CPL). Lo habitual en los textos es dar tres esquemas de axiomas proposicionales concretos (los axiomas de Hilbert) y obtener las tautologías como teoremas. Aquí seguimos el camino, también estándar [4, p. 33], de admitir directamente todas las tautologías clásicas; los propios autores lo describen como un “exceso axiomático” (*axiomatic overkill*) que prefieren a la alternativa de fijar unos pocos axiomas y derivar de ellos el resto, refinamiento que, a su juicio, tiene poco interés para sus propósitos, y lo mismo vale para los nuestros. Esta presentación evita desarrollar dentro de **K** toda la maquinaria del cálculo proposicional, que no es el objeto del trabajo. El precio es que el sistema deja de tener una base finita de axiomas y damos por supuesta la completitud del cálculo proposicional clásico, que no demostramos aquí.

En esa estrategia coincidimos con Blackburn, pero no en qué fórmulas cuentan como tautologías: su (CPL) admite las instancias de tautologías proposicionales, y el nuestro, las tautologías clásicas de la definición 3.2. Comprobemos que ambas clases coinciden:

En un sentido, toda instancia de una tautología proposicional es una tautología clásica. Si φ se obtiene de una tautología proposicional π sustituyendo sus variables por fórmulas modales, toda valoración booleana v de φ induce una valoración de las variables de π (a cada una se le da el valor $\hat{v}$ de la fórmula que la sustituye), y $\hat{v}(\varphi) = 1$ porque π es una tautología.

Recíprocamente, toda tautología clásica es una instancia de una tautología proposicional. Dada φ , sustituimos cada una de sus subfórmulas que empieza por $\Box$ y no está dentro de otra caja por una variable nueva, la misma para todas las ocurrencias de la misma subfórmula. El resultado π ya no contiene el operador $\Box$, y una valoración booleana de φ no es más que una valoración proposicional de π , con $v_{\Box}$ leída sobre las variables nuevas; luego si φ es una tautología clásica, π es una tautología proposicional y φ es una instancia suya.

Observación 3.5. Aparte de (CPL), el sistema de la definición 3.3 tampoco es literalmente el **K** de Blackburn [4, Def. 1.39], y es necesario dejar claro en qué difieren y por qué tienen los mismos teoremas. Allí, **K** cuenta con un tercer axioma, (Dual) $\Diamond p \leftrightarrow \neg\Box\neg p$, y con una tercera regla, la *sustitución uniforme*, que de $\vdash \varphi$ pasa a $\vdash \varphi^\sigma$, donde φ^σ resulta de sustituir uniformemente las variables de φ por fórmulas cualesquiera. Además, los axiomas (K) y (Dual) se enuncian con variables concretas p, q , y no como esquemas.

- (Dual) sobra en nuestro lenguaje. Como aquí $\Diamond\varphi$ es por definición $\neg\Box\neg\varphi$, el axioma se desplegaría a $\neg\Box\neg\varphi \leftrightarrow \neg\Box\neg\varphi$, una instancia de $\alpha \leftrightarrow \alpha$, que ya está en (CPL). Los propios autores lo advierten [4, p. 34]. El axioma solo hace falta porque su primitivo es $\Diamond$, y de haber tomado $\Box$ como primitivo no habría sido necesario. Podría objetarse que falta entonces la dualidad leída al revés, $\Box p \leftrightarrow \neg\Diamond\neg p$, que es la que en nuestro lenguaje dice algo. En efecto, al desplegar $\Diamond$ se convierte en $\Box p \leftrightarrow \neg\neg\Box\neg\neg p$, y sus dos subfórmulas modales maximales, $\Box p$ y $\Box\neg\neg p$, son átomos opacos distintos, de modo que no es una tautología clásica. Pero tampoco hay que postularla, porque es un teorema. De la tautología $p \leftrightarrow \neg\neg p$, la regla RE (proposición 3.7, más abajo) da $\vdash \Box p \leftrightarrow \Box\neg\neg p$, y la tautología $(\alpha \leftrightarrow \beta) \rightarrow (\alpha \leftrightarrow \neg\neg\beta)$ concluye con un *modus ponens*; el argumento vale igual con una fórmula cualquiera en lugar de p . La asimetría con Blackburn tiene una explicación sencilla. En su sistema $\Diamond$ solo aparece dentro del patrón $\neg\Diamond\neg$ con el que se define $\Box$, así que ningún axioma habla de un $\Diamond p$ desnudo y la dualidad hay que postularla; en el nuestro $\Box$ aparece libre en (K) y en (N), y eso basta para demostrarla.
- La *sustitución uniforme* sobra porque nuestros axiomas son esquemas. El axioma (K) se postula de golpe para todas las fórmulas φ, ψ , y el conjunto de las tautologías es cerrado bajo sustitución por su propia definición. Con ello, que el conjunto de teoremas sea cerrado bajo sustitución deja de ser una regla y pasa a ser un teorema, que se demuestra por inducción sobre la derivación.

Así pues, formulados en nuestro lenguaje, el sistema de Blackburn y el de la definición 3.3 demuestran exactamente las mismas fórmulas, ya que cada uno contiene los axiomas del otro y es cerrado bajo sus reglas. De hecho, nuestra definición “como la menor relación que cumple...” está más cerca de esa caracterización conjuntista que de la definición 1.39 de [4] (demostraciones como sucesiones finitas de fórmulas); la equivalencia entre ambos puntos de vista es el ejercicio 1.6.6 de [4], y la versión “menor relación” es precisamente la que produce una definición inductiva en Lean.

Cerrada la comparación con el libro, volvamos al contenido del sistema. Las únicas piezas propiamente modales de **K** son el axioma (K) y la regla (N), y entre las dos capturan el comportamiento mínimo de $\Box$: la necesidad distribuye sobre la implicación, y los teoremas son necesarios. Todo lo demás que **K** demuestra sale de combinarlas con el cálculo proposicional.

Conviene precisar el alcance de (N). La regla no afirma $\vdash \varphi \rightarrow \Box\varphi$ (las verdades contingentes no son necesarias), sino solo que las fórmulas demostrables pueden encajonarse. Blackburn dedica a este punto una advertencia expresa [4, Obs. 1.41]. Quien venga de la

deducción natural puede sentirse tentado de razonar así:

1. p hipótesis
2. $\Box p$ (N) aplicada a 1
3. $p \rightarrow \Box p$ se convierte la hipótesis en antecedente

El resultado no es válido, luego tampoco es un teorema de **K**. El error está en el paso 2: (N) solo se aplica a teoremas, y p es aquí una hipótesis. Volveremos sobre ello al definir la deducción desde hipótesis (sección 3.5).

De estos ingredientes se derivan reglas útiles. Destacamos las dos que formalizaremos, con sus demostraciones en papel, que después podrán compararse línea a línea con el código.

Proposición 3.6 (Regla de monotonía, RM; `provable_rm`). *Si $\vdash \varphi \rightarrow \psi$, entonces $\vdash \Box \varphi \rightarrow \Box \psi$.*

Demostración. Por (N), de $\vdash \varphi \rightarrow \psi$ obtenemos $\vdash \Box(\varphi \rightarrow \psi)$. El axioma (K) da $\vdash \Box(\varphi \rightarrow \psi) \rightarrow (\Box \varphi \rightarrow \Box \psi)$, y (MP) concluye. $\square$

Proposición 3.7 (Regla de equivalencia, RE; `provable_re`). *Si $\vdash \varphi \leftrightarrow \psi$, entonces $\vdash \Box \varphi \leftrightarrow \Box \psi$.*

Demostración. De $\vdash \varphi \leftrightarrow \psi$ se extraen, usando las tautologías $(\varphi \leftrightarrow \psi) \rightarrow (\varphi \rightarrow \psi)$ y $(\varphi \leftrightarrow \psi) \rightarrow (\psi \rightarrow \varphi)$ junto con (MP), las dos implicaciones $\vdash \varphi \rightarrow \psi$ y $\vdash \psi \rightarrow \varphi$. La regla RM da entonces $\vdash \Box \varphi \rightarrow \Box \psi$ y $\vdash \Box \psi \rightarrow \Box \varphi$, y la tautología $(\alpha \rightarrow \beta) \rightarrow ((\beta \rightarrow \alpha) \rightarrow (\alpha \leftrightarrow \beta))$, con dos aplicaciones de (MP), las reúne en la equivalencia buscada. $\square$

Obsérvese el patrón de estas dos demostraciones donde los únicos movimientos permitidos son citar un axioma y aplicar una regla.

3.4. Semántica de Kripke

La semántica estándar de la lógica modal, debida a Kripke [10], se apoya en una idea muy visual, la de *mundos posibles* conectados por una relación de accesibilidad. Una fórmula es verdadera o falsa en un mundo y los operadores modales cuantifican sobre los mundos accesibles desde el actual.

Definición 3.8 (Marcos y modelos). Un *marco de Kripke* es un par $F = (W, R)$ formado por un conjunto no vacío¹ W de mundos y una relación binaria $R \subseteq W \times W$, la *relación de accesibilidad*. Un *modelo de Kripke* sobre $\mathbf{VP}$ es un par $M = (F, V)$ donde F es un

¹En la formalización el tipo de mundo podrá ser vacío; véase la observación 4.1 en la sección 4.4.

marco y $V: W \rightarrow \mathcal{P}(\text{VP})$ es una *valoración* que indica qué variables son verdaderas en cada mundo.

Un marco es, sencillamente, un grafo dirigido; escribimos Rwv o wRv para indicar que “desde w es accesible v ”. En la valoración hay otra pequeña adaptación respecto a [4, Def. 1.19], donde $V: \text{VP} \rightarrow \mathcal{P}(W)$ asigna a cada variable el conjunto de mundos en que es verdadera, de modo que la cláusula atómica de la definición siguiente se lee allí $w \in V(p)$. Nosotros la escribimos al revés, $V: W \rightarrow \mathcal{P}(\text{VP})$, y leeremos $p \in V(w)$. La información es la misma (en ambos casos, un subconjunto de $W \times \text{VP}$).

Definición 3.9 (Satisfacción). Dado un modelo M y un mundo w , la relación $M, w \models \varphi$ (leído “ φ es verdadera en el mundo w de M ”) se define por recursión sobre φ :

$$\begin{aligned} M, w \models p & \iff p \in V(w), \\ M, w \models \perp & \text{no se da nunca,} \\ M, w \models \varphi \rightarrow \psi & \iff \text{si } M, w \models \varphi \text{ entonces } M, w \models \psi, \\ M, w \models \Box \varphi & \iff \text{para todo } v \text{ con } R w v, M, v \models \varphi \end{aligned}$$

La cláusula interesante es la última. La fórmula $\Box \varphi$ es verdadera en w cuando φ lo es en todos los mundos accesibles desde w . Desplegando las abreviaturas se comprueba que las conectivas derivadas reciben el significado esperado; en particular,

$$M, w \models \Diamond \varphi \iff \text{existe } v \text{ con } R w v \text{ y } M, v \models \varphi.$$

La posibilidad es, por tanto, la cuantificación existencial sobre los mundos accesibles.

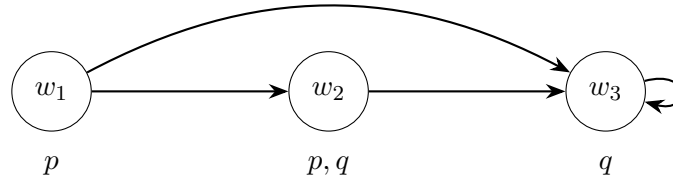


Figura 3.1: Un modelo de Kripke con tres mundos; bajo cada mundo se listan las variables que su valoración hace verdaderas. Se cumple, por ejemplo, $M, w_1 \models \Diamond p$ (hay un mundo accesible, w_2 , donde vale p). En cambio, $M, w_1 \not\models \Box p$, porque p falla en el mundo accesible w_3 . También $M, w_2 \models \Box q$, porque el único mundo accesible desde w_2 es w_3 , y allí vale q . En w_3 , cuyo único sucesor es él mismo, valen tanto $\Box q$ como $\Diamond q$; sin el bucle, w_3 no tendría sucesores y $\Box q$ seguiría siendo verdadera, pero vacuamente, mientras que $\Diamond q$ pasaría a ser falsa.

Observación 3.10. En un mundo sin sucesores, $\Box \varphi$ es verdadera vacuamente para cualquier φ (incluida $\perp$), pues la condición “en todo mundo accesible...” no tiene nada que

comprobar. Dualmente, $\Diamond\varphi$ es falsa para toda φ . Este fenómeno volverá a aparecer en la formalización: el modelo con el que probaremos la consistencia del sistema es, precisamente, un único mundo sin sucesores.

Definición 3.11 (Validez). Una fórmula φ es *válida en un modelo* M , escrito $M \models \varphi$, si $M, w \models \varphi$ para todo mundo w de M . Es *válida en un marco* F si es válida en todo modelo sobre F . Por último, es *válida*, escrito $\models \varphi$, si es válida en todo modelo de Kripke.

Observación 3.12. Una advertencia terminológica. Blackburn [4, Def. 1.21] llama a la primera noción *verdad global* (o *universal*) en el modelo, y reserva la palabra *válida* para los marcos y las clases de marcos [4, Def. 1.28]. El motivo es que la verdad global depende de la valoración elegida, y por eso no es una noción lógica. Si V hace verdadera p en todos los mundos, se tiene $M \models p$, pero eso informa sobre V y no sobre p . En ese mismo modelo puede fallar q , que se obtiene de p por sustitución uniforme [4, p. 34]. La validez en un marco cuantifica sobre todas las valoraciones, así que lo que queda depende solo de la relación de accesibilidad. Hemos preferido un único nombre para los tres niveles, que es también el que usa el código (`ValidInModel`, `ValidInFrame` y `Valid`), pero conviene tener presente la distinción al consultar el libro.

La noción central para nosotros es la última, porque es la contrapartida semántica de la demostrabilidad en **K**. La validez en un marco, que no depende de la valoración y que es la necesaria para la construcción de lógicas modales más fuertes. Por ejemplo, $\Box p \rightarrow p$ es válida en un marco exactamente cuando este es reflexivo, y añadirla como axioma da la lógica **T**.² En este trabajo definimos esta noción de validez para completar el planteamiento, aunque no la usaremos para formalizar la lógica **K**.

Ejemplo 3.13 (`valid_axiom_k`). El axioma K, $\Box(\varphi \rightarrow \psi) \rightarrow (\Box\varphi \rightarrow \Box\psi)$, es válido.

Sean M un modelo y w un mundo con $M, w \models \Box(\varphi \rightarrow \psi)$ y $M, w \models \Box\varphi$. Si v es accesible desde w , en v valen $\varphi \rightarrow \psi$ y φ , luego vale ψ . Como esto ocurre para todo mundo accesible desde w , se tiene $M, w \models \Box\psi$. Es un *modus ponens* mundo a mundo.

Ejemplo 3.14. La fórmula $\Box p \rightarrow p$ no es válida.

Tomemos el modelo con dos mundos $w R v$, sin más flechas, y con p verdadera solo en v . Entonces $M, w \models \Box p$, porque p vale en el único mundo accesible desde w , pero $M, w \not\models p$. Como **K** demostrará exactamente las fórmulas válidas, este contraejemplo muestra también que $\Box p \rightarrow p$ no es un teorema de **K**.

²De nuevo es esencial que p sea una variable: la instancia $\Box\top \rightarrow \top$ del esquema $\Box\varphi \rightarrow \varphi$ es válida en todo marco. Lo que caracteriza a los marcos reflexivos es la validez de $\Box p \rightarrow p$ o, equivalentemente, la de todas las instancias del esquema. Estas correspondencias entre fórmulas y propiedades de los marcos son el tema del capítulo 3 de [4].

3.5. Corrección y completitud: el mapa de la demostración

Tenemos dos nociones de *verdad*: la sintáctica ($\vdash \varphi$) y la semántica ($\models \varphi$). Los dos teoremas centrales de este trabajo afirman que coinciden.

Teorema 3.15 (Corrección; *soundness*). *Si $\vdash \varphi$, entonces $\models \varphi$.*

Teorema 3.16 (Completitud; *completeness*). *Si $\models \varphi$, entonces $\vdash \varphi$.*

Son los enunciados que formalizaremos en el capítulo 4. Es oportuno comentar que Blackburn demuestra algo más fuerte [4, Teorema 4.23], la llamada completitud *fuerte*. Esto es, que para todo conjunto de fórmulas Σ , si φ es consecuencia semántica de Σ (es decir, φ es verdadera en todo mundo de todo modelo en el que sean verdaderas todas las fórmulas de Σ), entonces φ se deduce de Σ en **K**. El teorema 3.16 es el caso $\Sigma = \emptyset$. En este trabajo nos limitamos a la versión débil; la construcción que describimos a continuación es, en cualquier caso, la misma que da la fuerte.

Corrección

De la corrección, que es la dirección sencilla, damos aquí un esbozo. Antes, incluimos un lema puente³, necesario:

Lema 3.17 (Lema puente; *cplEval_iff_satisfies*). *Sean M un modelo y w un mundo de M , y sea $v = (v_a, v_\Box)$ la valoración booleana dada por $v_a(p) = 1$ si y solo si $p \in V(w)$, y $v_\Box(\psi) = 1$ si y solo si $M, w \models \Box\psi$. Entonces, para toda fórmula φ ,*

$$\hat{v}(\varphi) = 1 \iff M, w \models \varphi.$$

Demostración. Por inducción estructural sobre φ . Para una variable p , ambos lados equivalen a $p \in V(w)$; para $\perp$, ambos son falsos; el caso $\varphi \rightarrow \psi$ sale de la hipótesis de inducción, porque las dos definiciones tratan la implicación igual; y el caso $\Box\psi$ se cumple por la propia definición de $v_\Box$, sin usar la hipótesis de inducción, que es exactamente lo que expresa la opacidad. $\square$

Esbozo de la demostración del teorema 3.15. Por inducción sobre la derivación de $\vdash \varphi$, con un caso por cada axioma y cada regla de la definición 3.3.

- El axioma (K) es válido, como se vio en el ejemplo 3.13.

³Este lema puente corresponde al ejercicio 1.3.4 de [4]. Es un buen ejemplo de las dificultades que presentan los asistentes de demostración: el paso que en papel apenas merece un ejercicio concentra en Lean buena parte del esfuerzo de la corrección.

- Si φ se obtiene por (MP) de $\psi \rightarrow \varphi$ y ψ , ambas válidas por hipótesis de inducción, fijemos un modelo M y un mundo w . En w valen $\psi \rightarrow \varphi$ y ψ , luego vale φ ; como M y w son arbitrarios, φ es válida.
- Si $\Box\psi$ se obtiene por (N) de ψ , válida por hipótesis de inducción, entonces ψ es verdadera en todos los mundos de todos los modelos, en particular en los accesibles desde cualquier w , luego $\Box\psi$ es válida.
- Queda el caso (CPL), que es el delicado, porque hay que conectar dos maneras muy distintas de evaluar una fórmula: la booleana de la definición 3.2, donde las subfórmulas $\Box\psi$ son átomos opacos, y la satisfacción en un mundo concreto. Recordemos que cada mundo determina una valoración booleana. Fijados M y w , tomemos $v_a(p) = 1$ si y solo si $p \in V(w)$, y $v_{\Box}(\psi) = 1$ si y solo si $M, w \models \Box\psi$. Por el lema 3.17, $\hat{v}(\varphi) = 1$ si y solo si $M, w \models \varphi$, para toda fórmula φ . Los casos de p , $\perp$ y $\varphi \rightarrow \psi$ son inmediatos, y el de $\Box\psi$ se cumple por la propia definición de $v_{\Box}$, sin usar la hipótesis de inducción, que es exactamente lo que expresa la opacidad. Ahora bien, si φ es una tautología clásica, $\hat{v}(\varphi) = 1$ para toda valoración booleana, en particular para esta, luego $M, w \models \varphi$. Como M y w eran arbitrarios, φ es válida.

□

Una consecuencia inmediata de la corrección es la *consistencia* del sistema ($\mathbf{k_consistent}$): $\not\models \perp$, pues $\perp$ falla en cualquier mundo de cualquier modelo (y existe al menos un modelo con un mundo).

Completitud

La completitud es sustancialmente más profunda, porque hay que fabricar, para cada fórmula no demostrable, un modelo que la refute. La estrategia clásica, que seguimos de [4], construye un *modelo canónico* que refuta simultáneamente todas las fórmulas no demostrables. Reunimos primero las piezas, dos definiciones y tres lemas, y con ellas la demostración del teorema se resuelve en unas líneas. Cada pieza se formaliza por separado en la sección 4.6.

Definición 3.18 (Deducibilidad desde hipótesis). Dado un conjunto de fórmulas Γ , la relación $\Gamma \vdash \varphi$ es la menor relación que cumple:

- (ax) si $\vdash \varphi$, entonces $\Gamma \vdash \varphi$;
- (hip) si $\varphi \in \Gamma$, entonces $\Gamma \vdash \varphi$;
- (MP) si $\Gamma \vdash \varphi \rightarrow \psi$ y $\Gamma \vdash \varphi$, entonces $\Gamma \vdash \psi$.

El símbolo $\vdash$ queda así sobrecargado, pero no se genera ambigüedad. $\emptyset \vdash \varphi$ si y solo si $\vdash \varphi$, porque (ax) da una dirección y en la otra la regla (hip) nunca puede aplicarse. En el código son dos relaciones distintas, `Provable` y `DeducibleFrom`, conectadas por `provable_of_deducibleFrom_empty`.

Aquí nos separamos de [4, Def. 4.4], que declara φ deducible de Γ cuando o bien $\vdash \varphi$, o bien $\vdash (\psi_1 \wedge \dots \wedge \psi_n) \rightarrow \varphi$ para ciertas $\psi_1, \dots, \psi_n \in \Gamma$. Ambas definiciones son equivalentes, y cada una tiene su ventaja. La de [4] hace inmediato el teorema de la deducción, que por eso no aparece enunciado en su libro. La nuestra evita manipular listas de fórmulas y conjunciones anidadas, y regala un principio de inducción sobre deducciones que usaremos repetidamente. A cambio, el teorema de la deducción pasa a requerir una demostración, la de la proposición 3.20.

Observación 3.19. La regla de necesitación no figura entre las tres. De $\Gamma \vdash \varphi$ no debe seguirse $\Gamma \vdash \Box\varphi$, porque las hipótesis pueden ser verdades contingentes, como vimos en la observación 1.41 de [4] que comentamos en la sección 3.3. Con la definición por conjunciones finitas el peligro no existe, porque (N) solo se aplica al teorema $\vdash (\psi_1 \wedge \dots \wedge \psi_n) \rightarrow \varphi$. Con la definición inductiva hay que excluirla a propósito, y la necesitación entra en las deducciones únicamente por la vía de (ax).

Proposición 3.20 (Teorema de la deducción; `deduction_theorem`). *Si $\Gamma \cup \{\varphi\} \vdash \psi$, entonces $\Gamma \vdash \varphi \rightarrow \psi$.*

Demostración. Por inducción sobre la deducción de ψ . Si ψ se obtuvo por (ax) o por (hip) con $\psi \in \Gamma$, la tautología $\psi \rightarrow (\varphi \rightarrow \psi)$ y (MP) dan lo buscado. Si se obtuvo por (hip) siendo ψ la propia φ , basta la tautología $\varphi \rightarrow \varphi$. Y si se obtuvo por (MP) de $\chi \rightarrow \psi$ y χ , la hipótesis de inducción da $\Gamma \vdash \varphi \rightarrow (\chi \rightarrow \psi)$ y $\Gamma \vdash \varphi \rightarrow \chi$, que la tautología $(\varphi \rightarrow (\chi \rightarrow \psi)) \rightarrow ((\varphi \rightarrow \chi) \rightarrow (\varphi \rightarrow \psi))$ combina con dos aplicaciones de (MP). $\square$

Las tres tautologías empleadas son la identidad y los axiomas A1 y A2 de Hilbert, que aquí no hemos tenido que postular porque (CPL) las incluye. Es una consecuencia del exceso axiomático de la sección 3.3.

Definición 3.21 (Consistencia y conjuntos maximales). Un conjunto de fórmulas Γ es *consistente* si $\Gamma \not\vdash \perp$. Es *maximal consistente* (en adelante, MCS, de *Maximal Consistent Set*) si es consistente y ningún conjunto consistente lo contiene estrictamente [4, Def. 4.15].

Proposición 3.22 (Los MCS describen mundos; `mcs_mem_of_deducible`, `mcs_mem_or_neg_mem`, `mcs_neg_mem_iff`, `mcs_imp_mem_iff`). *Sea Γ un MCS. Entonces Γ está deductivamente cerrado, es decir, $\Gamma \vdash \varphi$ implica $\varphi \in \Gamma$; para cada φ contiene exactamente una de φ y $\neg\varphi$; y contiene $\varphi \rightarrow \psi$ si y solo si contener φ lo obliga a contener ψ [4, Prop. 4.16].*

Estas tres propiedades son las que hacen de un MCS la “descripción completa de un mundo”. Esto es, que se comporta como una valoración total y coherente sobre todas las fórmulas, y son las que resolverán los casos proposicionales del lema de verdad.

Teorema 3.23 (Lema de Zorn, versión para familias de conjuntos; `zorn_subset_nonempty`, de Mathlib). *Sea $\mathcal{F}$ una familia de conjuntos tal que toda cadena no vacía $\mathcal{C} \subseteq \mathcal{F}$ (cadena respecto de la inclusión) admite una cota superior en $\mathcal{F}$, es decir, un $U \in \mathcal{F}$ con $\Delta \subseteq U$ para todo $\Delta \in \mathcal{C}$. Entonces, para cada $X \in \mathcal{F}$ existe un elemento maximal $\Delta \in \mathcal{F}$ con $X \subseteq \Delta$.*

Lema 3.24 (Lindenbaum; `lindenbaum`). *Todo conjunto consistente está contenido en un MCS.*

Demostración. Aplicamos el teorema 3.23 a la familia $\mathcal{F}$ de los conjuntos consistentes de fórmulas. La cota superior de una cadena no vacía $\mathcal{C}$ es su unión $\bigcup \mathcal{C}$, que es consistente: si $\bigcup \mathcal{C} \vdash \perp$, entonces, como toda deducción usa un número finito de hipótesis y $\mathcal{C}$ está totalmente ordenada por inclusión, todas ellas caben en un mismo eslabón $\Delta \in \mathcal{C}$, y $\Delta \vdash \perp$ contra la consistencia de Δ . Dado un consistente Γ , el teorema 3.23 devuelve un maximal de $\mathcal{F}$ que lo contiene, y ser maximal en la familia de los consistentes es exactamente ser un MCS. $\square$

La demostración de [4, Lema 4.17] es distinta puesto que enumera las fórmulas como $\varphi_0, \varphi_1, \dots$ y las va añadiendo una a una cuando no rompen la consistencia, lo que exige que el lenguaje sea numerable. La vía de Zorn elimina esa hipótesis, y de ahí que la definición 3.1 no imponga ninguna condición sobre VP.

Definición 3.25 (Modelo canónico). El *modelo canónico* M^c es el modelo de Kripke cuyos mundos son todos los MCS, con la accesibilidad

$$\Gamma R^c \Delta \iff \text{para toda } \psi, \Box\psi \in \Gamma \text{ implica } \psi \in \Delta,$$

y con la valoración que hace verdadera p en Γ exactamente cuando $p \in \Gamma$.

La accesibilidad canónica declara Δ accesible desde Γ cuando Δ cumple todas las obligaciones modales de Γ . Blackburn [4, Def. 4.18] la define con $\Diamond$, pidiendo que $\psi \in \Delta$ implique $\Diamond\psi \in \Gamma$, y prueba en su lema 4.19 que ambas formulaciones coinciden.

Lema 3.26 (Existencia; `existence_lemma`). *Sea Γ un MCS con $\Box\varphi \notin \Gamma$. Entonces existe un MCS Δ con $\Gamma R^c \Delta$ y $\varphi \notin \Delta$.*

Esbozo. Se aplica el lema 3.24 al conjunto $\{\psi : \Box\psi \in \Gamma\} \cup \{\neg\varphi\}$, de modo que el Δ resultante es accesible desde Γ por construcción y contiene $\neg\varphi$, luego no contiene φ . Lo

único que hay que comprobar es que ese conjunto es consistente, y el ingrediente clave es que si $\{\psi : \Box\psi \in \Gamma\} \vdash \varphi$, entonces $\Gamma \vdash \Box\varphi$. Esto significa que toda la deducción puede encajonarse usando el axioma (K) y la necesidad, y eso contradice $\Box\varphi \notin \Gamma$. $\square$

Es la forma dual del lema 4.20 de [4], que enuncia que si $\Diamond\varphi \in \Gamma$ existe un Δ accesible con $\varphi \in \Delta$. La demostración es la misma.

Lema 3.27 (Verdad; `truth_lemma`). *Para todo MCS Γ y toda fórmula φ se cumple $M^c, \Gamma \models \varphi$ si y solo si $\varphi \in \Gamma$ [4, Lema 4.21].*

Esbozo. Por inducción estructural sobre φ . El caso de las variables es la definición de la valoración canónica, y los casos de $\perp$ y de la implicación salen de la proposición 3.22. En el caso $\Box\varphi$, si $\Box\varphi \in \Gamma$ la definición de R^c pone φ en todo mundo accesible, y la hipótesis de inducción la hace verdadera allí. Recíprocamente, si $\Box\varphi \notin \Gamma$, el lema 3.26 da un mundo accesible donde φ no pertenece y, por hipótesis de inducción, no es verdadera. $\square$

En el modelo canónico, por tanto, verdad y pertenencia son lo mismo. Si φ falla en algún mundo (semántica), es que φ no esté en algún MCS (sintáctica). Con todo lo anterior, ya se puede proceder a la demostración de la completitud.

Demostración del teorema 3.16. Por contraposición. Supongamos $\not\models \varphi$; construimos un mundo del modelo canónico en el que φ falla.

El conjunto $\{\neg\varphi\}$ es consistente. Si no lo fuera, tendríamos $\{\neg\varphi\} \vdash \perp$, y la proposición 3.20 daría $\vdash \neg\varphi \rightarrow \perp$, que por definición de la negación es $\vdash \neg\neg\varphi$; con la tautología $\neg\neg\varphi \rightarrow \varphi$ y (MP) obtendríamos $\vdash \varphi$, contra la hipótesis.

Por el lema 3.24 existe un MCS Γ que contiene $\neg\varphi$. Como un MCS contiene exactamente una de φ y $\neg\varphi$ (proposición 3.22), se tiene $\varphi \notin \Gamma$, y el lema 3.27 da $M^c, \Gamma \not\models \varphi$.

Ese Γ es un mundo del modelo canónico donde φ no es verdadera, luego φ no es válida. $\square$

Es el argumento de los teoremas 4.22 y 4.23 de [4], reducido al caso de una sola fórmula. Nótese que el modelo canónico no depende de φ . Se construye una sola vez y refuta a la vez todas las fórmulas no demostrables, que es lo que hace viable la versión fuerte del teorema.

Capítulo 4

La formalización en Lean

Este capítulo recorre el código que constituye el núcleo del trabajo. No es un listado comentado exhaustivo. El fichero completo está en el repositorio del trabajo [12], y aquí se reproducen solo fragmentos sobre los que comentar. Cada definición se justifica antes de escribirse y cada demostración se compara con su versión en papel del capítulo 3; el orden es el mismo en los dos sitios.

4.1. La sintaxis

La gramática de la definición 3.1 se traduce en un tipo inductivo con un constructor por cláusula, y las abreviaturas de la sección 3.2, en definiciones ordinarias con su notación:

```
1 inductive ModalFormula (VP : Type) where
2   | atom : VP → ModalFormula VP           -- variable
3   | bot  : ModalFormula VP                 -- ⊥
4   | imp  : ModalFormula VP → ModalFormula VP → ModalFormula VP -- implicación
5   | box  : ModalFormula VP → ModalFormula VP -- □
6
7 def ModalFormula.neg (φ : ModalFormula VP) : ModalFormula VP :=
8   φ → ModalFormula.bot
9
10 def ModalFormula.and (φ ψ : ModalFormula VP) : ModalFormula VP :=
11   ~(φ → (~ψ))
12
13 def ModalFormula.dia (φ : ModalFormula VP) : ModalFormula VP :=
14   ~(□(~φ))
```

Esto es la correspondencia con $\varphi ::= p \mid \perp \mid \varphi \rightarrow \varphi \mid \Box \varphi$ que es “el menor conjunto que contiene las variables y $\perp$ y está cerrado bajo $\rightarrow$ y bajo $\Box$ ” (sección 2.2.3). De aquí

salen los dos principios que usaremos a lo largo de toda la implementación: el de recursión y el de inducción estructural.

Un detalle a observar es que $\sim\varphi$ sea $\varphi \rightarrow \text{ModalFormula.bot}$ imita deliberadamente la negación de Lean, $\neg p := p \rightarrow \text{False}$. La satisfacción de una negación modal será, sin necesidad de demostración, la negación de Lean de la satisfacción, y por eso `satisfies_neg` se cerrará con `rfl`.

4.2. El esquema de tautologías

La evaluación de la definición 3.2 es una recursión inmediata, y ser tautología, una doble cuantificación:

```

1 def cplEval (v_atom : VP → Prop) (v_box : ModalFormula VP → Prop) :
2   ModalFormula VP → Prop
3   | ModalFormula.atom x   => v_atom x
4   | ModalFormula.bot      => False
5   | ModalFormula.imp  $\varphi$   $\psi$  => cplEval v_atom v_box  $\varphi \rightarrow \text{cplEval v\_atom v\_box } \psi$ 
6   | ModalFormula.box  $\varphi$    => v_box  $\varphi$       -- la caja es un átomo opaco
7
8 def IsCPLTautology ( $\varphi$  : ModalFormula VP) : Prop :=
9    $\forall$  (v_atom : VP → Prop) (v_box : ModalFormula VP → Prop),
10    cplEval v_atom v_box  $\varphi$ 

```

Los argumentos `v_atom` y `v_box` son los v_a y $v_\square$ del papel. La diferencia de fondo es que $\hat{v}(\varphi)$ tomaba valores en $\{0,1\}$ y `cplEval` devuelve un `Prop`.

En el código no formalizamos ningún resultado general sobre tautologías, sino exactamente las ocho que el desarrollo necesita, recogidas en la tabla 4.1. Dos de ellas muestran el mecanismo entero:

```

1 theorem taut_id ( $\varphi$  : ModalFormula VP) : IsCPLTautology ( $\varphi \rightarrow \varphi$ ) := by
2   intro v_atom v_box h $\varphi$ 
3   exact h $\varphi$ 
4
5 theorem taut_dist ( $\varphi$   $\psi$   $\chi$  : ModalFormula VP) :
6   IsCPLTautology (( $\varphi \rightarrow (\psi \rightarrow \chi)$ )  $\rightarrow ((\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \chi))$ ) := by
7   intro v_atom v_box h $\varphi\psi\chi$  h $\varphi\psi$  h $\varphi$ 
8   exact h $\varphi\psi\chi$  h $\varphi$  (h $\varphi\psi$  h $\varphi$ )

```

El `intro` introduce los tres nombres de izquierda a derecha, igual que tres `intro` consecutivos. Cuando le llega el turno a `h $\varphi$` la meta es `cplEval v_atom v_box ( $\varphi \rightarrow \varphi$ )`,

que no *es* una implicación de Lean pero se reduce a una desplegando `cplEval`. Como las tácticas trabajan módulo igualdad definicional, la introducción funciona sin más. Dos valuaciones, una hipótesis y un `exact`. Es la misma demostración que se escribiría para $p \rightarrow p$ en lógica proposicional pura, consecuencia de haber evaluado en `Prop`. En `taut_dist` se aprecia mejor todavía, pues el término $\text{h}\varphi\psi\chi \text{ h}\varphi (\text{h}\varphi\psi \text{ h}\varphi)$ es la demostración entera del axioma A2 de Hilbert.

Nombre	Enunciado	Dónde se usa
<code>taut_id</code>	$\varphi \rightarrow \varphi$	teorema de deducción
<code>taut_weaken</code>	$\psi \rightarrow (\varphi \rightarrow \psi)$	teorema de deducción;
		<code>mcs_imp_mem_iff</code>
<code>taut_dist</code>	$(\varphi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \chi))$	teorema de deducción
<code>taut_dne</code>	$\neg\neg\varphi \rightarrow \varphi$	<code>consistent_insert_neg</code>
<code>taut_exfalso</code>	$\neg\varphi \rightarrow (\varphi \rightarrow \psi)$	<code>mcs_imp_mem_iff</code>
<code>taut_iff_mp</code>	$(\varphi \leftrightarrow \psi) \rightarrow (\varphi \rightarrow \psi)$	regla RE
<code>taut_iff_mpr</code>	$(\varphi \leftrightarrow \psi) \rightarrow (\psi \rightarrow \varphi)$	regla RE
<code>taut_iff_intro</code>	$(\varphi \rightarrow \psi) \rightarrow (\psi \rightarrow \varphi) \rightarrow (\varphi \leftrightarrow \psi)$	regla RE

Tabla 4.1: Las ocho tautologías clásicas demostradas y el lugar donde se emplean. Las tres primeras son la identidad y los axiomas A1 y A2 de Hilbert, que el teorema de la deducción 3.20 necesita.

La eliminación de la doble negación es la primera que necesita lógica clásica:

```

1 theorem taut_dne ( $\varphi$  : ModalFormula VP) : IsCPLTautology ( $\neg\neg\varphi \rightarrow \varphi$ ) := by
2   intro v_atom v_box hnn
3   -- hnn : (cplEval v_atom v_box  $\varphi \rightarrow \text{False}$ )  $\rightarrow \text{False}$ 
4   --  $\vdash \text{cplEval v_atom v_box } \varphi$ 
5   by_contra hn
6   -- hn : cplEval v_atom v_box  $\varphi \rightarrow \text{False}$ 
7   --  $\vdash \text{False}$ 
8   exact hnn hn

```

El estado hace visible la definición de la negación: `cplEval v_atom v_box ( $\neg\varphi$ )` no es un primitivo, sino `cplEval v_atom v_box  $\varphi \rightarrow \text{False}$` . Una doble negación es esa construcción anidada dos veces.

Las tres tautologías del bicondicional, en cambio, se dificultan por nuestro lenguaje mínimo. Desplegando definiciones, si P abrevia la evaluación de $\varphi \rightarrow \psi$ y Q la de $\psi \rightarrow \varphi$, lo que `cplEval` produce a partir de $\varphi \leftrightarrow \psi$ es $\neg(P \rightarrow \neg Q)$, que es correcto pero incómodo de manipular.

```

1 theorem taut_iff_mp ( $\varphi \psi$  : ModalFormula VP) :

```

```

2   IsCPLTautology (( $\varphi \longleftrightarrow \psi$ )  $\longrightarrow$  ( $\varphi \longrightarrow \psi$ )) := by
3   -- Abreviamos  $P := \text{eval } (\varphi \longrightarrow \psi)$  y  $Q := \text{eval } (\psi \longrightarrow \varphi)$ 
4   intro v_atom v_box hiff
5   --  $\text{hiff} : \neg(P \rightarrow \neg Q) \vdash P$ 
6   by_contra hnimp
7   --  $\text{hnimp} : \neg P \vdash \text{False}$ 
8   apply hiff
9   --  $\vdash P \rightarrow \neg Q$ 
10  intro himp
11  --  $\text{himp} : P \vdash \neg Q$ 
12  contradiction

```

La meta al llegar a `contradiction` es $\neg Q$, y nunca se demuestra. Basta con que `himp` y `hnimp` se contradigan. Las otras dos tautologías del bicondicional siguen el mismo esquema.

4.3. El sistema axiomático

La definición 3.3 describe $\vdash \varphi$ como la menor relación cerrada bajo dos axiomas y dos reglas. Eso es una definición inductiva:

```

1  inductive Provable : ModalFormula VP  $\rightarrow$  Prop where
2  | cpl ( $\varphi$  : ModalFormula VP) (h : IsCPLTautology  $\varphi$ ) : Provable  $\varphi$ 
3  | mp { $\varphi \psi$  : ModalFormula VP} (h $\varphi\psi$  : Provable ( $\varphi \longrightarrow \psi$ )) (h $\varphi$  : Provable  $\varphi$ ) :
4    Provable  $\psi$ 
5  | k ( $\varphi \psi$  : ModalFormula VP) : Provable ( $\Box(\varphi \longrightarrow \psi) \longrightarrow (\Box\varphi \longrightarrow \Box\psi)$ )
6  | nec { $\varphi$  : ModalFormula VP} (h : Provable  $\varphi$ ) : Provable ( $\Box\varphi$ )

```

Esto es el uso de los tipos inductivos anunciado en la sección 2.2.3. `Provable` es una familia de proposiciones indexada por fórmulas, y un término de tipo `Provable  $\varphi$` es un árbol finito de aplicaciones de axiomas y reglas, es decir, una derivación en **K**. De ahí salen a la vez las cuatro reglas como constructores y el principio de inducción sobre derivaciones, que es lo que hace que la corrección, más adelante, quepa en cuatro líneas. Los tres teoremas `provable_iff_mp`, `provable_iff_mpr` y `provable_iff_intro`, de una línea cada uno, se limitan a convertir teoremas de **K** las tautologías del bicondicional mediante `Provable.cpl`.

Las proposiciones 3.6 y 3.7 muestran el parecido entre una demostración en papel y una en Lean:

```

1  theorem provable_rm { $\varphi \psi$  : ModalFormula VP} (h $\varphi\psi$  : Provable ( $\varphi \longrightarrow \psi$ )) :

```

```

2   Provable ( $\Box\varphi \rightarrow \Box\psi$ ) := by
3   have hnec : Provable ( $\Box(\varphi \rightarrow \psi)$ ) := Provable.nec h $\varphi\psi$            -- necesidad
4   have hK : Provable ( $\Box(\varphi \rightarrow \psi) \rightarrow (\Box\varphi \rightarrow \Box\psi)$ ) := Provable.k  $\varphi\psi$  -- axioma K
5   exact Provable.mp hK hnec                                           -- modus ponens

```

La demostración en papel de la proposición 3.6 son tres frases y aquí son tres líneas, en el mismo orden. La correspondencia es el resultado de usar `have` en lugar de un único término anidado. Una versión compacta, `Provable.mp (Provable.k  $\varphi\psi$ ) (Provable.nec h $\varphi\psi$ )`, sería correcta, ocupa una línea y es la que aparecería en una biblioteca, pero hay que leerla de dentro afuera para reconstruir el argumento.

```

1  theorem provable_re { $\varphi\psi$  : ModalFormula VP} (h : Provable ( $\varphi \leftrightarrow \psi$ )) :
2    Provable ( $\Box\varphi \leftrightarrow \Box\psi$ ) := by
3    -- Extraemos  $\vdash \varphi \rightarrow \psi$  y aplicamos RM
4    have hmp : Provable ( $(\varphi \leftrightarrow \psi) \rightarrow (\varphi \rightarrow \psi)$ ) := provable_iff_mp  $\varphi\psi$ 
5    have h $\varphi\psi$  : Provable ( $\varphi \rightarrow \psi$ ) := Provable.mp hmp h
6    have hbox $\varphi\psi$  : Provable ( $\Box\varphi \rightarrow \Box\psi$ ) := provable_rm h $\varphi\psi$ 
7    -- Extraemos  $\vdash \psi \rightarrow \varphi$  y aplicamos RM
8    have hmpr : Provable ( $(\varphi \leftrightarrow \psi) \rightarrow (\psi \rightarrow \varphi)$ ) := provable_iff_mpr  $\varphi\psi$ 
9    have h $\psi\varphi$  : Provable ( $\psi \rightarrow \varphi$ ) := Provable.mp hmpr h
10   have hbox $\psi\varphi$  : Provable ( $\Box\psi \rightarrow \Box\varphi$ ) := provable_rm h $\psi\varphi$ 
11   -- Reensamblamos el bicondicional con dos modus ponens
12   have hintro : Provable ( $(\Box\varphi \rightarrow \Box\psi) \rightarrow (\Box\psi \rightarrow \Box\varphi) \rightarrow (\Box\varphi \leftrightarrow \Box\psi)$ ) :=
13     provable_iff_intro ( $\Box\varphi$ ) ( $\Box\psi$ )
14   have hstep : Provable ( $(\Box\psi \rightarrow \Box\varphi) \rightarrow (\Box\varphi \leftrightarrow \Box\psi)$ ) := Provable.mp hintro hbox $\varphi\psi$ 
15   exact Provable.mp hstep hbox $\psi\varphi$ 

```

En `provable_re`, los tres comentarios marcan los tres movimientos de la proposición 3.7, y cada `have` lleva su tipo escrito porque, leídos en columna, son la lista de teoremas intermedios del argumento en papel. Los dos últimos pasos hacen visible algo que allí queda tras la frase “con dos aplicaciones de (MP)”: como en **K** no hay regla de introducción de la conjunción, los dos antecedentes de `taut_iff_intro` se consumen de uno en uno.

Hay que subrayar en qué sentido RM y RE son *reglas*. No son constructores de `Provable`, que sigue teniendo cuatro, sino teoremas *sobre* `Provable` demostrados en el metalenguaje. Son reglas admisibles y no primitivas. En Lean es la distinción que hay entre estar dentro o fuera del bloque `inductive`.

4.4. La semántica de Kripke

La semántica es donde más se nota que Lean no es teoría de conjuntos. El conjunto de mundos de la definición 3.8 se convierte en un tipo, la relación en una función con valores en `Prop`, y la satisfacción de la definición 3.9 en una recursión estructural:

```
1 structure KripkeFrame where
2   World : Type
3   R : World → World → Prop
4
5 structure KripkeModel (VP : Type) extends KripkeFrame where
6   V : World → VP → Prop
7
8 def Satisfies (M : KripkeModel VP) (w : M.World) : ModalFormula VP → Prop
9   | ModalFormula.atom x   => M.V w x
10  | ModalFormula.bot       => False
11  | ModalFormula.imp φ ψ   => Satisfies M w φ → Satisfies M w ψ
12  | ModalFormula.box φ     => ∀ v : M.World, M.R w v → Satisfies M v φ
```

Que `World` sea un `Type` y no un `Set` es forzoso. En Lean todo conjunto lo es de términos de un tipo fijado de antemano, de modo que declarar los mundos como un conjunto exigiría disponer ya de un tipo previo del que fueran términos, y aquí no hay ninguno. Se escribe `Set α`, y sus elementos son términos de α . La consecuencia se verá en la subsección 4.6.4, donde habrá que fabricar un tipo cuyos elementos sean exactamente los MCS.

Observación 4.1. No exigimos que `World` sea no vacío, a diferencia de la definición 3.8. Sobre el modelo (único) con tipo de mundos vacío toda fórmula es válida vacuamente, lo cual es inofensivo. La validez cuantifica sobre *todos* los modelos, y sobra con que exista alguno no vacío para que $\perp$ no sea válida. A cambio, las definiciones quedan más limpias (sin arrastrar una prueba de no vacuidad) y los teoremas, marginalmente más generales.

En `Satisfies`, las tres primeras cláusulas coinciden con las de `cp1Eval` salvo por la valoración empleada, y la cuarta es la única que cambia. Donde la evaluación clásica devolvía un valor opaco, la satisfacción abre la caja y cuantifica sobre los mundos accesibles. Toda la diferencia entre la lógica proposicional y la modal cabe en esa línea, y que las otras tres coincidan es lo que hará posible el lema puente. Justifica además la elección de $\Box$ como primitivo (sección 3.2) puesto que su semántica es un universal, y un universal en Lean es una función, que se usa aplicándola y se demuestra con `intro`, mientras que la cláusula dual de $\Diamond$ sería un existencial, que exige exhibir un testigo para construirlo y descomponerlo para usarlo.

Veamos ahora que las conectivas derivadas reciben el significado esperado. El primero

es:

```
1 theorem satisfies_neg (M : KripkeModel VP) (w : M.World) (p : ModalFormula VP) :
2   Satisfies M w (~p) ↔ ¬ Satisfies M w p := by
3   rfl    -- ambos lados se despliegan a Satisfies M w p → False
```

Una sola táctica, y de las que no hacen nada porque los dos lados son definicionalmente iguales, que es consecuencia de haber imitado la negación de Lean. El segundo, `satisfies_neg`, dice que `Satisfies M w (◇p)` equivale a que exista un mundo accesible donde valga `p`, y es el caso opuesto. Su lado izquierdo se despliega a

```
(∀ v, M.R w v → (Satisfies M v p → False)) → False,
```

y pasar de ahí al existencial es la inferencia $\neg\forall x \neg P(x) \Rightarrow \exists x P(x)$, no demostrable intuicionistamente.

Los tres niveles de la definición 3.11 se declaran a continuación:

```
1 def ValidInModel (M : KripkeModel VP) (φ : ModalFormula VP) : Prop :=
2   ∀ w : M.World, Satisfies M w φ
3
4 def ValidInFrame (F : KripkeFrame) (φ : ModalFormula VP) : Prop :=
5   ∀ V : F.World → VP → Prop, ValidInModel { toKripkeFrame := F, V := V } φ
6
7 def Valid (φ : ModalFormula VP) : Prop :=
8   ∀ M : KripkeModel VP, ValidInModel M φ
```

La segunda, `ValidInFrame`, no se usa después, pero completa la definición 3.11 y es la que haría falta para seguir hacia lógicas más fuertes.

4.5. El teorema de corrección

La corrección es la dirección sencilla, y su demostración por inducción sobre derivaciones ocupa cuatro líneas. Casi todo el trabajo está en el lema puente 3.17, que en el capítulo 3 se demostraba en cinco renglones y que aquí es el resultado más laborioso de la sección. Es un buen ejemplo del desajuste entre el esfuerzo en papel y el formal, y el propio Blackburn lo deja como ejercicio.

```
1 theorem cplEval_iff_satisfies (M : KripkeModel VP) (w : M.World)
2   (φ : ModalFormula VP) :
3   cplEval (fun x => M.V w x) (fun ψ => Satisfies M w (□ψ)) φ ↔
4   Satisfies M w φ := by
```

```

5 induction  $\varphi$  with
6 | atom  $x \Rightarrow$  rfl
7 | bot  $\Rightarrow$  rfl
8 | imp  $\varphi \psi$  ih $\varphi$  ih $\psi \Rightarrow$ 
9   -- El objetivo es  $(A_1 \rightarrow B_1) \leftrightarrow (A_2 \rightarrow B_2)$  con  $ih\varphi : A_1 \leftrightarrow A_2$  e  $ih\psi : B_1 \leftrightarrow B_2$ 
10  apply Iff.intro
11  · intro h h $\varphi$ 
12    exact ih $\psi$ .mp (h (ih $\varphi$ .mpr h $\varphi$ ))
13  · ...
14 | box  $\varphi$  ih  $\Rightarrow$ 
15   -- cplEval ( $\Box\varphi$ ) es v_box  $\varphi$ , elegida como Satisfies  $M w (\Box\varphi)$ : no se usa ih
16  rfl

```

De los cuatro casos, tres se cierran con `rfl`. El de la caja es el significativo: que sea `rfl` quiere decir que `v_box` se eligió para que lo fuera, pero además la hipótesis de inducción no se usa. En papel, la demostración del lema 3.17 dice “el caso $\Box\psi$ se cumple por la propia definición de $v_\Box$, sin usar la hipótesis de inducción, que es exactamente lo que expresa la opacidad”. En Lean esa frase se vuelve un hecho verificable, pues el nombre `ih` aparece en el patrón y no vuelve a aparecer.

La opacidad de la caja, que era una observación conceptual, se ha convertido en una hipótesis no utilizada. El único caso con trabajo es la implicación, y su dificultad es combinatoria. Hay que dividir el bicondicional y componer las hipótesis de inducción en el sentido adecuado en cada dirección, el detalle que en papel se resuelve con la palabra “análogamente”. Nótese que la inducción no necesita **generalizing**, porque el modelo y el mundo están fijados desde el enunciado; es la diferencia con el lema de verdad.

Para ver que las tautologías clásicas son válidas, basta con instanciar la hipótesis, que es un $\forall$ sobre valoraciones, en las dos extraídas de `w`, y aplicar el lema puente. Los otros tres casos son cortos:

```

1 theorem valid_axiom_k ( $\varphi \psi$  : ModalFormula VP) :
2   Valid ( $\Box(\varphi \rightarrow \psi) \rightarrow (\Box\varphi \rightarrow \Box\psi)$ ) := by
3   intro M w hboximp hbox $\varphi$  v hRv
4   exact hboximp v hRv (hbox $\varphi$  v hRv)

```

Los seis nombres del primer `intro` recorren de una vez los seis niveles del enunciado, hasta el mundo accesible que introduce la caja del consecuente; el “*modus ponens* mundo a mundo” del ejemplo 3.13 es un párrafo del papel en una línea.

```

1 theorem valid_nec { $\varphi$  : ModalFormula VP} (h $\varphi$  : Valid  $\varphi$ ) : Valid ( $\Box\varphi$ ) := by
2   intro M w v hRv

```

```
3 exact hφ M v
```

En `valid_nec` conviene mirar qué *no* se usa. Si nos fijamos, la meta se cierra sin mencionar `w` ni `hRv`, porque la validez vale en todos los mundos y da igual desde dónde se llegue. Ese “da igual desde dónde” es la razón por la que la necesitación preserva la validez pero no la verdad en un mundo, la advertencia de la sección 3.3 y la observación 3.19.

Con las piezas separadas, el teorema de corrección es un ensamblaje:

```
1 theorem soundness {φ : ModalFormula VP} (h : Provable φ) : Valid φ := by
2   induction h with
3   | cpl φ htaut      => exact valid_of_cpl_tautology htaut
4   | mp hφψ hφ ihφψ ihφ => exact valid_mp ihφψ ihφ
5   | k φ ψ           => exact valid_axiom_k φ ψ
6   | nec h ih        => exact valid_nec ih
```

La táctica `induction h` abre un caso por constructor, es decir, uno por axioma y por regla. En los de las reglas, Lean proporciona tanto las subderivaciones como las hipótesis de inducción, y la demostración usa solo estas últimas, porque lo que hace falta es su validez. El esquema “los axiomas son válidos y las reglas preservan la validez, luego todo teorema es válido” está escrito con esas cuatro líneas y nada más.

El sistema no demuestra que $\perp$ no es `Provable`, por lo que incluimos el siguiente corolario. Para verlo hace falta exhibir un modelo concreto, y basta el más pobre posible: un solo mundo, ninguna flecha y ninguna variable verdadera. El tipo `Unit` de Lean tiene exactamente un término, escrito `()`; tomarlo como tipo de mundos da ese universo de un único punto, y `()` será el mundo en el que evaluemos.

```
1 def trivialModel (VP : Type) : KripkeModel VP where
2   World := Unit
3   R := fun _ _ => False
4   V := fun _ _ => False
5
6 theorem k_consistent : ¬ Provable (ModalFormula.bot : ModalFormula VP) := by
7   intro hprov
8   have hsat : Satisfies (trivialModel VP) () ModalFormula.bot :=
9     soundness hprov (trivialModel VP) ()
10  exact hsat -- Satisfies _ _ ⊥ es False por definición
```

Observamos que el cierre de la demostración es particular. La meta es `False` y se cierra con una hipótesis cuyo tipo es una satisfacción. No hay ningún paso oculto, porque la

segunda cláusula de `Satisfies` dice que la satisfacción de `ModalFormula.bot` es `False`. Una demostración de que $\perp$ es verdadera en algún mundo es, simplemente, una demostración de lo imposible. Y aquí es donde hacía falta un modelo con al menos un mundo, según la observación 4.1.

4.6. El teorema de completitud

La completitud ocupa aproximadamente la mitad del código del fichero porque, a diferencia de la corrección, tenemos que fabricar un modelo a partir de material puramente sintáctico. El desarrollo se organiza en las seis subsecciones que siguen.

4.6.1. Deducción a partir de hipótesis

La relación $\Gamma \vdash \varphi$ de la definición 3.18 se declara, igual que `Provable`, como un tipo inductivo:

```

1 inductive DeducibleFrom ( $\Gamma$  : Set (ModalFormula VP)) :
2   ModalFormula VP  $\rightarrow$  Prop where
3   | ax { $\varphi$  : ModalFormula VP} (h : Provable  $\varphi$ ) : DeducibleFrom  $\Gamma$   $\varphi$ 
4   | mem { $\varphi$  : ModalFormula VP} (h :  $\varphi \in \Gamma$ ) : DeducibleFrom  $\Gamma$   $\varphi$ 
5   | mp { $\varphi$   $\psi$  : ModalFormula VP} (h $\varphi\psi$  : DeducibleFrom  $\Gamma$  ( $\varphi \rightarrow \psi$ ))
6     (h $\varphi$  : DeducibleFrom  $\Gamma$   $\varphi$ ) : DeducibleFrom  $\Gamma$   $\psi$ 

```

El conjunto Γ va antes de los dos puntos, como parámetro, y la fórmula después, como índice. Un parámetro permanece fijo en todos los constructores y en el principio de inducción, mientras que un índice puede variar, que es justo lo que hace falta, pues en una deducción las hipótesis no cambian de un paso a otro pero la fórmula deducida sí.

Lo más importante de la declaración es lo que *no* contiene. No hay constructor para la necesidad. Su ausencia se razonaba en la observación 3.19. Admitir que de $\Gamma \vdash \varphi$ se sigue $\Gamma \vdash \Box\varphi$ sería un error, porque las hipótesis pueden ser verdades contingentes. Como decíamos, con la definición de Blackburn por conjunciones finitas el error es imposible por construcción, con una definición inductiva hay que excluirlo a mano, y la necesidad entra en las deducciones solo por la vía de `ax`.

Dos lemas preliminares son inducciones de tres líneas. La monotonía (`deducibleFrom_mono`) rehace la misma deducción sobre un conjunto mayor, y solo el caso `mem` hace algo. El otro, `provable_of_deducibleFrom_empty`, establece la mitad no trivial de la coincidencia $\emptyset \vdash \varphi \iff \vdash \varphi$ que la sección 3.5 daba por descontada al sobrecargar el símbolo $\vdash$; su caso `mem` es el único uso de `simp` en todo el fichero, para descartar la hipótesis imposible $\varphi \in \emptyset$.

La proposición 3.20 es el primer resultado con contenido, y el modelo de todas las inducciones sobre derivaciones que vendrán después:

```

1 theorem deduction_theorem {Γ : Set (ModalFormula VP)} {φ ψ : ModalFormula VP}
2   (h : DeducibleFrom (insert φ Γ) ψ) : DeducibleFrom Γ (φ → ψ) := by
3   induction h with
4   | @ax χ hχ =>
5     -- χ es un teorema: lo debilitamos con la tautología χ → (φ → χ)
6     have hweak : DeducibleFrom Γ (χ → (φ → χ)) :=
7       DeducibleFrom.ax (Provable.cpl _ (taut_weaken χ φ))
8     exact DeducibleFrom.mp hweak (DeducibleFrom.ax hχ)
9   | @mem χ hχ =>
10    -- χ es hipótesis de insert φ Γ: o es la propia φ, o ya estaba en Γ
11    rcases Set.mem_insert_iff.mp hχ with heq | hmem
12    · subst heq -- χ = φ: basta la identidad φ → φ
13      exact DeducibleFrom.ax (Provable.cpl _ (taut_id _))
14    · ... -- χ ∈ Γ: se repite el debilitamiento del caso ax
15  | @mp χ ξ _ _ ihχξ ihχ =>
16    -- ihχξ : Γ ⊢ φ → (χ → ξ) e ihχ : Γ ⊢ φ → χ; la tautología A2 las combina
17    have hdist : DeducibleFrom Γ ((φ → (χ → ξ)) → ((φ → χ) → (φ → ξ)))
18    :=
19      DeducibleFrom.ax (Provable.cpl _ (taut_dist φ χ ξ))
20    exact DeducibleFrom.mp (DeducibleFrom.mp hdist ihχξ) ihχ

```

La arroba que aparece en cada caso, por ejemplo en `| @ax χ hχ` es la notación de la sección 2.3. El patrón necesita nombrar la fórmula, que el constructor declara implícita, y no puede llamarla ψ porque ese nombre ya está tomado por el enunciado. Los tres casos reproducen los tres párrafos de la demostración de la proposición 3.20; obsérvese el primero, donde se encadenan tres niveles en una sola expresión: una tautología clásica, convertida a teorema de **K** por (CPL), convertida a deducción desde Γ por (ax).

El caso `mem` se ramifica y deja ver algo característico del trabajo de formalización: en papel, la proposición 3.20 despacha juntos los casos (ax) y (hip) con $\psi \in \Gamma$, porque en ambos se dispone de $\Gamma \vdash \psi$ sin importar de dónde venga, mientras que en Lean son casos distintos por venir de constructores distintos y el mismo bloque de tres líneas hay que escribirlo dos veces.

4.6.2. Consistencia y conjuntos maximales consistentes

La definición 3.21 da lugar a dos declaraciones:

```

1 def Consistent (Γ : Set (ModalFormula VP)) : Prop :=

```

```

2  ¬ DeducibleFrom Γ ModalFormula.bot
3
4  def MaximalConsistent (Γ : Set (ModalFormula VP)) : Prop :=
5  Consistent Γ ∧ ∀ Δ : Set (ModalFormula VP), Consistent Δ → Γ ⊆ Δ → Δ ⊆ Γ

```

La primera ilustra por qué se separan las notaciones de los dos lenguajes. El $\neg$ es del metalenguaje y afirma que cierta deducción no existe, mientras que `ModalFormula.bot` es un dato y, de usarse con los mismos símbolos, la definición parecería decir algo sobre la fórmula $\neg\perp$.

La segunda expresa la parte “ningún consistente lo contiene estrictamente” por el camino contrapuesto, de la definición 3.21. Esta forma de expresarlo es la que usa el lema de Zorn de Mathlib que devuelve la maximalidad, de modo que el resultado de Zorn encajará sin traducir nada. En lo que sigue, `hΓ.1` es “ Γ es consistente” y `hΓ.2`, “ Γ es maximal”.

Que el vacío es consistente se sigue en dos líneas de `k_consistent`. El siguiente lema es pequeño y se usa en los dos momentos decisivos del argumento:

```

1  theorem consistent_insert_neg {Γ : Set (ModalFormula VP)} {φ : ModalFormula VP}
2    (h : ¬ DeducibleFrom Γ φ) : Consistent (insert (¬φ) Γ) := by
3    intro hbot
4    -- Teorema de deducción:  $\Gamma \vdash \sim\varphi \rightarrow \perp$ , que por definición es  $\sim\sim\varphi$ 
5    have hdneg : DeducibleFrom Γ (¬¬φ) := deduction_theorem hbot
6    have hdne : DeducibleFrom Γ (¬¬φ → φ) :=
7      DeducibleFrom.ax (Provable.cpl _ (taut_dne φ))
8    exact h (DeducibleFrom.mp hdne hdneg)

```

Es interesante detenernos en el uso del primer `have`, en la línea 5. El término `deduction_theorem hbot` no tiene el tipo declarado, sino que tiene `DeducibleFrom Γ (¬φ → ModalFormula.bot)`. Lean lo acepta porque los dos son definicionalmente iguales tomando $\alpha := \sim\varphi$ en $\sim\alpha := \alpha \rightarrow \text{ModalFormula.bot}$. Escribir el tipo en la forma que tiene sentido matemático hace legible la demostración y así el código dice lo que uno quiere decir y Lean comprueba que coincide con lo que hay.

Las tres propiedades de la proposición 3.22, las que convierten un MCS en la descripción completa de un mundo, se demuestran a continuación. La primera demuestra que Γ está deductivamente cerrado:

```

1  theorem mcs_mem_of_deducible {Γ : Set (ModalFormula VP)}
2    (hΓ : MaximalConsistent Γ) {φ : ModalFormula VP} (h : DeducibleFrom Γ φ) :
3    φ ∈ Γ := by
4    -- Si insert φ Γ dedujera ⊥, el teorema de deducción daría  $\Gamma \vdash \varphi \rightarrow \perp$ , que

```

```

5  -- junto con  $h : \Gamma \vdash \varphi$  da  $\Gamma \vdash \perp$ , contra la consistencia de  $\Gamma$ , que es  $h\Gamma.1$ 
6  have hcons : Consistent (insert  $\varphi$   $\Gamma$ ) := by
7    intro hbot
8    have himp : DeducibleFrom  $\Gamma$  ( $\varphi \longrightarrow \text{ModalFormula.bot}$ ) := deduction_theorem hbot
9    exact h $\Gamma.1$  (DeducibleFrom.mp himp h)
10 -- Por maximalidad ( $h\Gamma.2$ ),  $\text{insert } \varphi \Gamma \subseteq \Gamma$ . En Mathlib un  $\subseteq$  es una función de
11 -- pertenencias, así que se aplica a  $\varphi \in \text{insert } \varphi \Gamma$  y devuelve  $\varphi \in \Gamma$ 
    (Set.mem_insert  $\varphi \Gamma$ )

```

Junto con el constructor `DeducibleFrom.mem`, que da la implicación recíproca, este lema establece que para un MCS pertenecer y ser deducible son la misma cosa que es lo que permite tratarlo como una valoración. Las otras dos propiedades siguen el mismo patrón y sus enunciados son

```

theorem mcs_mem_or_neg_mem ... ( $\varphi : \text{ModalFormula VP}$ ) :  $\varphi \in \Gamma \vee \sim\varphi \in \Gamma$ 
theorem mcs_neg_mem_iff ... ( $\varphi : \text{ModalFormula VP}$ ) :  $\sim\varphi \in \Gamma \leftrightarrow \varphi \notin \Gamma$ 
theorem mcs_imp_mem_iff ... : ( $\varphi \longrightarrow \psi$ )  $\in \Gamma \leftrightarrow (\varphi \in \Gamma \rightarrow \psi \in \Gamma)$ 

```

El primero es una distinción de casos con `by_cases` sobre si Γ deduce φ . El uso de esa táctica que anunciaba la sección 2.5 es que la disyunción no viene de ninguna hipótesis previa sino del tercero excluido.

El segundo la convierte en excluyente, y su dirección directa vuelve a aprovechar la definición de la negación, pues tener $\sim\varphi$ y φ en Γ permite un *modus ponens* que produce $\Gamma \vdash \perp$ sin ningún paso intermedio.

El tercero es el que usará el lema de verdad. Su demostración emplea las dos tautologías que faltaban: la dirección directa es un *modus ponens* seguido de la consideración de que Γ es deductivamente cerrado, y la recíproca se ramifica según φ pertenezca o no, construyendo la implicación con `taut_weaken` a partir del consecuente en un caso y con `taut_exfalso` a partir de $\sim\varphi$ en el otro. Son los dos casos en que una implicación es verdadera, aquí manifestados como los dos casos de un `by_cases`.

4.6.3. El lema de Lindenbaum, vía Zorn

Como decíamos en la sección 3.2 demostración no procede de [4], que sigue la vía de la enumeración, sino que es la aplicación estándar del lema de Zorn a la familia de los conjuntos consistentes; la hemos redactado a la medida de la versión que ofrece Mathlib (sección 2.6), y de ahí la hipótesis de que las cadenas sean no vacías, que en papel suele omitirse.

Aplicar Zorn exige comprobar que la unión de una cadena de conjuntos consistentes es consistente. En papel el argumento es una frase: “si $\bigcup \mathcal{C} \vdash \perp$, entonces, como toda

deducción usa un número finito de hipótesis y $\mathcal{C}$ está totalmente ordenada por inclusión, todas ellas caben en un mismo eslabón”. Esa frase esconde un hecho que hay que demostrar, que nosotros debemos aislar como lema previo:

```

1 theorem deducibleFrom_sUnion_chain {c : Set (Set (ModalFormula VP))}
2   (hchain : IsChain (· ⊆ ·) c) (hne : c.Nonempty) {φ : ModalFormula VP}
3   (h : DeducibleFrom (⋃₀ c) φ) : ∃ Δ ∈ c, DeducibleFrom Δ φ := by
4   induction h with
5   | @ax χ hχ =>
6     -- Un teorema se deduce desde cualquier eslabón, y la cadena no es vacía
7     obtain ⟨Δ, hΔ⟩ := hne
8     exact ⟨Δ, hΔ, DeducibleFrom.ax hχ⟩
9   | @mem χ hχ => ... -- la hipótesis pertenece a algún eslabón
10  | @mp χ ξ _ _ ihχξ ihχ =>
11    -- Cada premisa vive en su propio eslabón...
12    obtain ⟨Δ₁, hΔ₁, hd₁⟩ := ihχξ
13    obtain ⟨Δ₂, hΔ₂, hd₂⟩ := ihχ
14    -- ...y, al estar encadenados, ambas caben en el mayor de los dos
15    rcases eq_or_ne Δ₁ Δ₂ with rfl | hneq
16    · exact ⟨_, hΔ₁, DeducibleFrom.mp hd₁ hd₂⟩
17    · rcases hchain hΔ₁ hΔ₂ hneq with hsub | hsub
18      · exact ⟨Δ₂, hΔ₂, DeducibleFrom.mp (deducibleFrom_mono hsub hd₁) hd₂⟩
19      · exact ⟨Δ₁, hΔ₁, DeducibleFrom.mp hd₁ (deducibleFrom_mono hsub hd₂)⟩

```

Es un punto del código donde la formalización muestra las diferencias entre demostrar en papel y en Lean. Si nos fijamos, en ninguna parte se habla de finitud. La demostración en papel invoca que “toda deducción usa un número finito de hipótesis”, un hecho que requiere a su vez una inducción sobre derivaciones y una definición del conjunto de hipótesis efectivamente usadas. Aquí se sustituye por una inducción directa en la que cada caso produce el eslabón que hace falta: en `ax` sirve cualquiera, en `mem` el que contiene la hipótesis, y en `mp` hay dos, uno por premisa, que la cadena garantiza comparados y la monotonía funde en uno. La finitud del razonamiento informal está repartida a lo largo del árbol de derivación.

Dos detalles más. El caso `ax` no puede concluir sin exhibir algún eslabón, lo que obliga a pedir `hne : c.Nonempty` y explica por qué la versión de Zorn de Mathlib exige que las cadenas sean no vacías. Además `IsChain` compara elementos distintos, de modo que antes de usarla hay que descartar con `eq_or_ne` que los dos eslabones coincidan. En papel nadie se plantea ese caso, porque “el mayor de los dos” incluye tácitamente que puedan ser el mismo. En Lean, se debe ser explícito.

Con ese lema, Lindenbaum es la aplicación de Zorn:

```

1 theorem lindenbaum {Γ : Set (ModalFormula VP)} (hΓ : Consistent Γ) :
2   ∃ Δ : Set (ModalFormula VP), Γ ⊆ Δ ∧ MaximalConsistent Δ := by
3   -- Condición de cadenas que exige zorn_subset_nonempty
4   have hchains : ∀ c ⊆ {Θ : Set (ModalFormula VP) | Consistent Θ},
5     IsChain (· ⊆ ·) c → c.Nonempty →
6     ∃ ub ∈ {Θ : Set (ModalFormula VP) | Consistent Θ}, ∀ Δ ∈ c, Δ ⊆ ub := by
7     intro c hcS hchain hne
8     refine ⟨⋃₀ c, ?_, fun Δ hΔ => Set.subset_sUnion_of_mem hΔ⟩
9     intro hbot
10    obtain ⟨Δ, hΔc, hΔbot⟩ := deducibleFrom_sUnion_chain hchain hne hbot
11    exact hcS hΔc hΔbot
12  obtain ⟨Δ, hΓΔ, hΔmax⟩ :=
13    zorn_subset_nonempty {Θ : Set (ModalFormula VP) | Consistent Θ} hchains Γ hΓ
14  -- El maximal que devuelve Zorn es exactamente un MCS según la definición
15  refine ⟨Δ, hΓΔ, hΔmax.1, ?_⟩
16  intro Θ hΘ hsubΘ
17  exact hΔmax.2 hΘ hsubΘ

```

La demostración tiene tres partes. La primera es el `have hchains` de la línea 4, que comprueba lo que `zorn_subset_nonempty` pide para poder aplicarse: que toda cadena no vacía de conjuntos consistentes tiene una cota superior que también es consistente. Su enunciado está copiado del que aparece en el teorema de Mathlib (sección 2.6), cambiando la familia genérica por la nuestra. Dentro, el `refine` propone la unión de la cadena como cota superior y demuestra sobre la marcha lo inmediato, que cada eslabón está contenido en ella. Queda por ver que esa unión es consistente, y de eso se encarga el lema anterior en una línea.

La segunda parte es la llamada a Zorn (línea 13), que devuelve un conjunto Δ que contiene a Γ y es maximal dentro de la familia.

La tercera comprueba que ese Δ es lo que buscábamos, y es donde se aprovecha la forma en que definimos `MaximalConsistent` en la subsección 4.6.2. El predicado `Maximal` de Mathlib es una conjunción de dos afirmaciones: que Δ pertenece a la familia, es decir, que es consistente, y que cualquier conjunto consistente que lo contenga está contenido en él. las dos que pide nuestra definición, de modo que la primera se entrega tal cual con `hΔmax.1`. La segunda necesita un `intro` y un `exact` por un motivo técnico, que Mathlib deja implícito el conjunto cuantificado y nosotros lo escribimos explícito.

4.6.4. El modelo canónico y el lema de encajonado

Los mundos del modelo canónico son los conjuntos maximales consistentes, y traducir eso exige un tipo cuyos elementos sean exactamente esos conjuntos: el subtipo.

```

1 def canonicalModel (VP : Type) : KripkeModel VP where
2   World := {  $\Gamma$  : Set (ModalFormula VP) // MaximalConsistent  $\Gamma$  }
3   R := fun  $\Gamma$   $\Delta$  =>  $\forall \psi$  : ModalFormula VP,  $\Box\psi \in \Gamma.val \rightarrow \psi \in \Delta.val$ 
4   V := fun  $\Gamma$  x => ModalFormula.atom x  $\in \Gamma.val$ 

```

Los elementos de un subtipo son pares de un valor y una demostración, contruidos con $\langle \Gamma, h\Gamma \rangle$ y descompuestos con `.val` y `.property`. Un mundo canónico no es, pues, un conjunto de fórmulas, sino un conjunto *acompañado de la demostración* de que es maximal consistente. Es un coste de notación pero el beneficio es que la maximalidad está siempre disponible sin pasarla como hipótesis.

Para el lema de existencia necesitamos que las deducciones se puedan “encajonar”. El esbozo del lema 3.26 lo resolvía con “toda la deducción puede encajonarse usando el axioma (K) y la necesidad”. De nuevo, en Lean cada regla se encajona por separado:

```

1 theorem deducibleFrom_box { $\Gamma$  : Set (ModalFormula VP)} { $\varphi$  : ModalFormula VP}
2   (h : DeducibleFrom { $\psi$  : ModalFormula VP |  $\Box\psi \in \Gamma$ }  $\varphi$ ) :
3   DeducibleFrom  $\Gamma$  ( $\Box\varphi$ ) := by
4   induction h with
5   | @ax  $\chi$  h $\chi$  =>
6     --  $\chi$  es un teorema de K: por necesidad,  $\Box\chi$  también lo es
7     exact DeducibleFrom.ax (Provable.nec h $\chi$ )
8   | @mem  $\chi$  h $\chi$  =>
9     -- h $\chi$  dice, leído con cuidado, exactamente que  $\Box\chi \in \Gamma$ 
10    exact DeducibleFrom.mem h $\chi$ 
11   | @mp  $\chi$   $\xi$  _ _ ih $\chi\xi$  ih $\chi$  =>
12     have hK : DeducibleFrom  $\Gamma$  ( $\Box(\chi \rightarrow \xi) \rightarrow (\Box\chi \rightarrow \Box\xi)$ ) :=
13       DeducibleFrom.ax (Provable.k  $\chi$   $\xi$ )
14     exact DeducibleFrom.mp (DeducibleFrom.mp hK ih $\chi\xi$ ) ih $\chi$ 

```

El caso `ax` es el único lugar de toda la completitud donde se usa la necesidad, y es lícito precisamente porque `ax` solo admite teoremas de **K**. El caso `mem` es engañosamente breve, pues la hipótesis y la meta encajan sin ningún paso. Esto ocurre porque la notación de conjuntos por comprensión es una función, $\{\psi \mid P \psi\}$ es `fun  $\psi$  => P  $\psi$` , y la pertenencia es la aplicación. El caso `mp` es donde se ve para qué sirve el axioma (K) pues sin él, el encajonado no atravesaría el *modus ponens*.

4.6.5. Los lemas de existencia y de verdad

El lema de existencia 3.26 construye, para cada MCS que no contenga $\Box\varphi$, un mundo accesible donde φ falla:

```

1 theorem existence_lemma {Γ : Set (ModalFormula VP)} (hΓ : MaximalConsistent Γ)
2   {φ : ModalFormula VP} (hbox : □φ ∉ Γ) :
3   ∃ Δ : Set (ModalFormula VP),
4     MaximalConsistent Δ ∧ (∀ ψ : ModalFormula VP, □ψ ∈ Γ → ψ ∈ Δ) ∧
5     φ ∉ Δ := by
6   -- (1) φ no se deduce de las obligaciones (encajonado más clausura deductiva)
7   have hnφ : ¬ DeducibleFrom {ψ : ModalFormula VP | □ψ ∈ Γ} φ := by
8     intro hdφ
9     exact hbox (mcs_mem_of_deducible hΓ (deducibleFrom_box hdφ))
10  -- (2) Añadir ~φ a las obligaciones conserva la consistencia
11  have hcons : Consistent (insert (~φ) {ψ : ModalFormula VP | □ψ ∈ Γ}) :=
12    consistent_insert_neg hnφ
13  -- (3) Lindenbaum extiende ese conjunto a un MCS Δ
14  obtain ⟨Δ, hsub, hΔ⟩ := lindenbaum hcons
15  refine ⟨Δ, hΔ, ?_, ?_⟩
16  ... -- las dos comprobaciones restantes, rutinarias

```

Nótese que el enunciado no menciona el modelo canónico. La accesibilidad va desplegada y los conjuntos sin empaquetar en subtipos, para que el lema sea puramente sintáctico y no haya que arrastrar `.val` y `.property` por una demostración. Los tres pasos comentados son el esbozo del lema 3.26 en el mismo orden, y el `refine` escribe directamente la parte “creativa”, el testigo del existencial, dejando como huecos dos comprobaciones rutinarias: que Δ contiene las obligaciones de Γ , porque ya estaban en el conjunto que Lindenbaum extendió, y que $\varphi \notin \Delta$, porque $\sim\varphi \in \Delta$.

El lema de verdad es el resultado central de la sección y el punto donde converge todo lo anterior:

```

1 theorem truth_lemma (φ : ModalFormula VP) (Γ : (canonicalModel VP).World) :
2   Satisfies (canonicalModel VP) Γ φ ↔ φ ∈ Γ.val := by
3   induction φ generalizing Γ with
4   | atom x => exact Iff rfl -- por definición de la valoración canónica
5   | bot => ... -- ⊥ ni se satisface ni cabe en un consistente
6   | imp φ ψ ihφ ihψ =>
7     -- Ambos lados son implicaciones; las hipótesis de inducción las cruzan
8     constructor
9     · intro hsat
10      apply (mcs_imp_mem_iff Γ.property).mpr
11      intro hφ
12      exact (ihψ Γ).mp (hsat ((ihφ Γ).mpr hφ))
13    · intro hmem hφ
14      exact (ihψ Γ).mpr ((mcs_imp_mem_iff Γ.property).mp hmem ((ihφ Γ).mp hφ))

```

```

15 | box  $\varphi$  ih =>
16 |   constructor
17 |   · -- ( $\Rightarrow$ ) Por reducción al absurdo, con el lema de existencia
18 |     intro hsat
19 |     by_contra hbox
20 |     obtain  $\langle \Delta, h\Delta mcs, hR, hn\varphi \rangle :=$  existence_lemma  $\Gamma$ .property hbox
21 |     exact hn $\varphi$  ((ih  $\langle \Delta, h\Delta mcs \rangle$ ).mp (hsat  $\langle \Delta, h\Delta mcs \rangle$  hR))
22 |   · -- ( $\Leftarrow$ ) La accesibilidad canónica reparte  $\varphi$  a todo mundo accesible
23 |     intro hbox  $\Delta$  hR
24 |     exact (ih  $\Delta$ ).mpr (hR  $\varphi$  hbox)

```

Lo primero es explicar `generalizing`, cuya justificación aplazamos desde la sección 2.5. La inducción es sobre la fórmula, pero el enunciado depende también del mundo, y el mundo cambia dentro de la inducción. En el caso de la caja hay que aplicar la hipótesis de inducción en un Δ distinto del Γ de partida. Con `induction  $\varphi$` a secas, esa hipótesis quedaría fijada al Γ del contexto y solo diría “para esta subfórmula y *este* mundo”. Añadiendo `generalizing  $\Gamma$` , Lean saca el mundo del contexto antes de aplicar el principio de inducción y las hipótesis quedan cuantificadas universalmente sobre él. Por eso en el cuerpo se escriben $ih\varphi \Gamma$, $ih\psi \Gamma$, $ih \Delta$. Es lo análogo a enunciar la hipótesis de inducción “para todo mundo” en papel.

El caso de las variables es `Iff.rfl`, y el de $\perp$ usa la consistencia del mundo, `$\Gamma$ .property.1`, que no hay que suponer porque viene incorporada en el subtipo. El de la implicación es la razón por la que se demostró `mcs_imp_mem_iff`. Las hipótesis de inducción conectan cada extremo con el suyo y ese lema conecta la implicación de un lado con la del otro, de modo que ambas direcciones caben en dos líneas cada una.

El caso de la caja es el núcleo. La dirección de derecha a izquierda es inmediata. Si $\Box\varphi \in \Gamma$, la definición de R pone φ en todo mundo accesible y la hipótesis de inducción la hace verdadera allí. Esto explica por qué la relación canónica se define como se define. La dirección contraria es por contraposición: si $\Box\varphi$ fuese verdadera en Γ sin pertenecerle, el lema de existencia daría un MCS accesible sin φ , pero `hsat` lo hace verdadero allí y la hipótesis de inducción convierte esa verdad en pertenencia. La expresión `$\langle \Delta, h\Delta mcs \rangle$` empaqueta el conjunto que devuelve el lema de existencia junto con su maximalidad para convertirlo en un mundo.

4.6.6. Completitud y adecuación

Con el lema de verdad, la demostración del teorema 3.16 es la del capítulo 3 traducida paso a paso:

```

1 | theorem completeness { $\varphi$  : ModalFormula VP} (h : Valid  $\varphi$ ) : Provable  $\varphi$  := by

```

```

2  by_contra hnφ
3  -- (1) φ no se deduce sin hipótesis...
4  have hnd : ¬ DeducibleFrom (∅ : Set (ModalFormula VP)) φ := by
5    intro hd
6    exact hnφ (provable_of_deducibleFrom_empty hd)
7  -- (2) ...luego {~φ} es consistente...
8  have hcons : Consistent (insert (~φ) (∅ : Set (ModalFormula VP))) :=
9    consistent_insert_neg hnd
10 -- (3) ...y Lindenbaum lo extiende a un MCS Γ con ~φ ∈ Γ
11 obtain ⟨Γ, hsub, hΓ⟩ := lindenbaum hcons
12 have hnφΓ : ~φ ∈ Γ := hsub (Set.mem_insert _ _)
13 -- (4) Γ es un mundo canónico; como φ es válida, se satisface en él
14 have hsat : Satisfies (canonicalModel VP) ⟨Γ, hΓ⟩ φ :=
15   h (canonicalModel VP) ⟨Γ, hΓ⟩
16 -- (5) El lema de verdad convierte esa satisfacción en φ ∈ Γ
17 have hφΓ : φ ∈ Γ := (truth_lemma φ ⟨Γ, hΓ⟩).mp hsat
18 -- (6) Pero φ y ~φ juntas producen Γ ⊢ ⊥, contra la consistencia de Γ
19 exact hΓ.1 (DeducibleFrom.mp (DeducibleFrom.mem hnφΓ) (DeducibleFrom.mem hφΓ))

```

Los seis pasos comentados se corresponden con los tres párrafos de la demostración del teorema 3.16. Nos detenemos en el paso (4). La hipótesis $h : \text{Valid } \varphi$ es, desplegada, una función que a cada modelo y cada mundo asigna una demostración de que φ se satisface allí. Todo lo que hay que hacer para usar la validez es aplicarla al modelo canónico y al mundo $\langle \Gamma, h\Gamma \rangle$. Es la única vez en toda la demostración en que interviene la hipótesis del teorema. Toda la construcción anterior existe para poder escribir esa línea.

Finalmente, los dos teoremas, corrección y completitud, se reúnen en una línea:

```

theorem provable_iff_valid (φ : ModalFormula VP) : Provable φ ↔ Valid φ :=
  ⟨soundness, completeness⟩

```

Si leemos el enunciado con calma vemos que, a la izquierda tenemos un tipo inductivo cuyos términos son árboles de derivación contruidos con cuatro reglas; a la derecha, una cuantificación sobre todos los modelos de Kripke, todos los mundos y todas las valoraciones. Son dos objetos sin ningún parecido, y el teorema afirma que tienen exactamente los mismos elementos. Que Lean lo haya verificado significa que entre las dos definiciones no quedan pasos sin justificar, ni analogías, ni casos omitido por simetría.

Queda una observación sobre lo que el teorema *no* dice. Lo demostrado es la completitud *débil*, esto es, que toda fórmula válida es demostrable. La versión *fuerte*, que es la que establece [4, Teorema 4.23] y que recordábamos en la sección 3.5, admite además partir de premisas. Si φ es consecuencia semántica de un conjunto Σ de fórmulas, entonces $\Sigma \vdash \varphi$.

Nuestro teorema es el caso $\Sigma = \emptyset$. La versión fuerte no exige rehacer nada, simplemente usa el mismo modelo canónico y los mismos lemas. Falta definir la consecuencia semántica y comprobar que el MCS que Lindenbaum construye a partir de Σ satisface todas sus fórmulas. La infraestructura está construida, y en el capítulo 6 esto se recoge entre las continuaciones naturales del trabajo.

Capítulo 5

Hacia la lógica epistémica: el problema de los niños embarrados

La idea de este capítulo es aplicar la formalización abstracta de la lógica modal a un ejemplo. Usaremos un acertijo clásico de razonamiento sobre el conocimiento, el llamado problema de los *niños embarrados* (*muddy children*). Presentaremos el problema y algunas de las preguntas que plantea y haremos una modelización coherente con el desarrollo de los capítulos anteriores con la que demostrar la solución.

5.1. El acertijo

Un enunciado posible del problema, adaptado del de Fagin, Halpern, Moses y Vardi [8, §1.1], es el siguiente:

Hay un grupo de n niños jugando en el jardín. Cuando terminan de jugar, hay $k \geq 1$ de ellos que tienen su frente manchada de barro. Un adulto se acerca al jardín y reúne a los n niños en círculo, de forma que todos los niños pueden ver las frentes de los demás pero no la suya propia.¹ Posteriormente anuncia:

“Al menos uno de vosotros tiene la frente manchada.”

A continuación pregunta:

“¿Sabe alguno de vosotros, con certeza, si su propia frente está manchada?”

Los niños que lo sepan deben decir “sí” en voz alta, todos a la vez; los demás guardan silencio. Si alguien dice que sí, el juego termina. Si nadie dice nada, el adulto repite la

¹Se entiende que un niño no puede saber si su frente está manchada inicialmente. No puede tocársela o notarse el barro en la frente.

misma pregunta de forma sucesiva (como si fueran turnos o rondas) hasta que algún niño responda. ¿Pueden los niños, sin intercambiar más información que sus propias respuestas, llegar a la conclusión de si su frente está o no manchada?

Si suponemos que

- todos los niños son lógicos perfectos y capaces de realizar cualquier deducción válida a partir de la información que se presenta;
- cada niño es sincero, entendiendo sinceridad como silencio salvo certeza deductiva: un niño dice “sí” exactamente cuando ha deducido el estado de su propia frente, y calla en caso contrario;
- existe simultaneidad en la recepción de información. Todos oyen a la vez y todos responden a la vez en cada “turno de pregunta”;
- y, todo lo anterior, junto con el enunciado del problema, es *conocimiento común* entre los niños: todos lo saben, todos saben que todos lo saben, todos saben que todos saben que todos lo saben, y así sucesivamente,

entonces, la respuesta al acertijo es que sí pueden llegar a una conclusión sobre su frente y, en concreto:

En las primeras $k - 1$ rondas ningún niño dirá nada. En la ronda k , los k niños manchados dirán que sí, y los limpios seguirán en silencio.

El último de los supuestos merece atención, y volveremos sobre él en la sección 5.6: no basta con que todos los niños sepan las reglas, hace falta la torre de conocimiento de “todos saben que todos saben...”. Por el momento, los primeros casos pueden verse a mano.

- Si $k = 1$, el único niño manchado ve todo el resto de frentes limpias. Como el adulto ha dicho que hay al menos una manchada y no es ninguna de las que ve, tiene que ser la suya, y contesta que sí en la primera ronda.
- Si $k = 2$, cada uno de los dos niños manchados ve una sola frente sucia y ninguno sabe nada de la suya, así que en la primera ronda nadie dice nada. Pero entonces cada uno razona: “si yo estuviera limpio, el otro manchado no habría visto ninguna frente manchada y habría contestado que sí. Como ha callado, yo estoy manchado” y ambos contestan que sí en la segunda ronda.

- Con $k = 3$ el argumento se repite un nivel más arriba: cada niño manchado ve dos frentes sucias, espera que esos dos contesten que sí en la segunda ronda y, al ver que callan, deduce que él también está manchado.

Esta idea será la que formalizaremos en el resto del capítulo.

Además del razonamiento hacia la solución, el problema deja una paradoja aparente para cualquier caso con $k \geq 2$. En cualquier supuesto así, cada niño ve al menos una frente con barro, de modo que antes de que el adulto informase sobre la existencia de alguna frente manchada, todos los niños ya lo sabían. El anuncio del adulto no parece informar a nadie de nada. Sin embargo, el anuncio es necesario. Sin él, en la situación con $k = 2$ el razonamiento del primer niño no se iniciaría, porque no hay ninguna razón para que el segundo hubiera dicho nada en la primera ronda aunque el primero estuviera limpio. La respuesta a esta paradoja está en el último de los supuestos que hemos escrito, y profundizaremos en ella en la sección 5.6, cuando tengamos el modelo construido. Veremos que existen diferencias entre conocimiento mutuo y conocimiento común.

El acertijo circula en muchas versiones (damas con la cara tiznada, sombreros de colores, sabios y sellos, esposas infieles). Una de las más antiguas del siglo XX es la de Littlewood [11, pp. 3–4], planteada con tres damas que viajan en un vagón de tren y que además generaliza a n damas y resuelve el caso general por inducción. El tratamiento estándar, con la terminología de los niños embarrados, es el del capítulo 1 de [8]. La lectura dinámica que seguimos aquí, en la que cada anuncio transforma el modelo, es la de [16].

5.2. El conocimiento como modalidad

La tabla 3.1 del capítulo 3 anunciaba la lectura epistémica de $\Box$: “ φ se sabe”. Pero saber es algo que alguien hace. A ese “alguien”, en el contexto de la lógica epistémica, se le conoce como *agente*.

Fijemos entonces un agente. Sus mundos posibles son las descripciones completas de cómo podría ser la realidad, y la relación de accesibilidad se lee ahora como una relación de *indistinguibilidad*: $w \sim v$ significa que, si v fuera el estado real del mundo, el agente tendría exactamente las mismas observaciones que tiene en w , de modo que no puede diferenciarlo y, por ende, descartarlo. Con esa lectura, la cláusula de la definición de satisfacción 3.9,

$$M, w \models \Box\varphi \iff \text{para todo } v \text{ con } w \sim v, M, v \models \varphi,$$

dice que φ es verdadera en todos los estados que el agente considera posibles, es decir, que φ se sigue de lo que observa. Esto es lo que llamamos *saber* φ . La fórmula dual, $\Diamond\varphi$,

dice que hay un estado compatible con sus observaciones en el que φ es cierta. Esto es que el agente *considera posible* φ . En la lectura epistémica es habitual escribir $K\varphi$ y $\widehat{K}\varphi$ (de *knowledge*) en lugar de $\Box\varphi$ y $\Diamond\varphi$, pero se trata de los mismos operadores.

Cuando hay varios agentes, cada uno tiene su propia relación de indistinguibilidad sobre los mismos mundos. El tratamiento estándar las reúne en un solo modelo multiagente, con una relación $\sim_i$ y un operador K_i por agente, sobre un lenguaje con tantas cajas como agentes [8, cap. 2]. Nosotros no podemos hacer exactamente eso, porque tanto el lenguaje del capítulo 3 como su formalización del capítulo 4 tienen una sola caja. Pero obsérvese qué es realmente un modelo multiagente: un modelo para un tipo de semejanza con n diamantes es una tupla $(W, \sim_1, \dots, \sim_n, V)$ [4, Def. 1.23], esto es, *una familia de modelos de Kripke con los mismos mundos y la misma valoración, uno por agente..* Y una fórmula del fragmento monoagente, esto es, una en la que aparece el operador de un único agente, se evalúa enteramente dentro del modelo de ese agente. Por eso, en todo este capítulo, en lugar de definir un lenguaje nuevo escribiremos

$$M^{(i)}, S \models \Box\varphi$$

para decir que el niño i sabe φ , donde $M^{(i)}$ es el modelo de Kripke cuya relación de accesibilidad es la indistinguibilidad de i . El lenguaje, los modelos y la satisfacción son los de los capítulos 3 y 4, sin ningún cambio.

El problema que trae esta decisión es que no se pueden escribir fórmulas que mezclen el conocimiento de dos niños, como $K_1K_2\varphi$. Para el teorema 5.10 no hace falta. Sí será necesario, en cambio, para el anuncio de la sección 5.4 y para la discusión del conocimiento común de la sección 5.6. En ambos casos lo señalaremos explícitamente y explicaremos cómo se suple.

5.3. El modelo del acertijo

Para construir el modelo hay que decidir tres cosas: qué es un mundo posible, cuándo dos mundos son indistinguibles para cada niño y qué variables son verdaderas en cada mundo. Un mundo debe describir por completo lo que puede variar. Pero lo único que varía es quién está manchado, por tanto, un mundo es un subconjunto de $N = \{1, \dots, n\}$, y hay 2^n . El niño i ve todas las frentes menos la suya, de modo que no puede distinguir dos mundos que coincidan en todo salvo en él mismo. Y la variable m_i , “el niño i está manchado”, es verdadera en el mundo S exactamente cuando $i \in S$.

Definición 5.1 (El cubo). Sea $N = \{1, \dots, n\}$ y sea $VP = \{m_1, \dots, m_n\}$. Para cada

niño i , sea $M^{(i)} = (W, \sim_i, V)$ el modelo de Kripke dado por

$$W = \mathcal{P}(N), \quad S \sim_i T \iff S \setminus \{i\} = T \setminus \{i\}, \quad V(S) = \{m_j : j \in S\}.$$

Los mundos y la valoración son los mismos para todos los niños. Lo que cambia de un modelo a otro es la relación de accesibilidad. El *mundo real* S^* es el conjunto de los niños que están efectivamente manchados, y $k = |S^*|$.

El nombre de *cubo* viene de la forma del modelo. Si identificamos cada subconjunto con su vector característico, los mundos son los vértices del cubo $\{0, 1\}^n$ y las aristas de la dirección i son los pares relacionados por $\sim_i$. La figura 5.1 lo dibuja para $n = 3$, superponiendo los tres modelos en un solo diagrama.

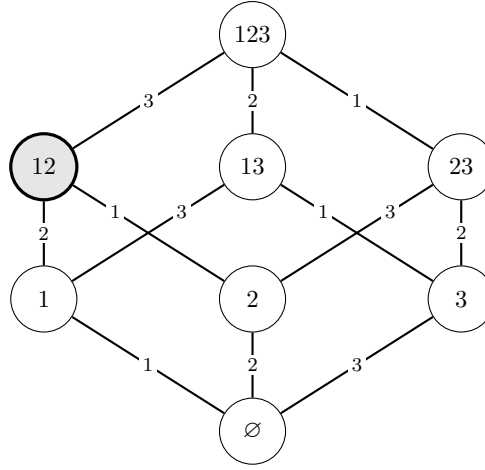


Figura 5.1: Los modelos $M^{(1)}$, $M^{(2)}$ y $M^{(3)}$ para $n = 3$, dibujados a la vez. Cada mundo se nombra por el conjunto de niños manchados (12 abrevia $\{1, 2\}$) y cada arista lleva la etiqueta del niño que no distingue sus extremos, de modo que el modelo del niño i es el que resulta de quedarse solo con las aristas etiquetadas con i . Se omiten los bucles $S \sim_i S$. Los mundos están dispuestos por niveles según su número de manchados; en gris, el mundo real del ejemplo que seguiremos, $S^* = \{1, 2\}$.

El lema siguiente dice que las clases de indistinguibilidad son muy pequeñas. Es un lema necesario para nuestra definición particular de $\sim_i$ mediante $S \setminus \{i\} = T \setminus \{i\}$. Cada niño duda entre dos mundos y solo dos. Escribimos $S \triangle \{i\}$ (la diferencia simétrica de S con el conjunto $\{i\}$) para el mundo que resulta de cambiar en S el estado del niño i , esto es, $S \setminus \{i\}$ cuando $i \in S$ y $S \cup \{i\}$ cuando $i \notin S$. Obsérvese que $S \triangle \{i\}$ es siempre distinto de S .

Lema 5.2 (Vecinos). $S \sim_i T$ si y solo si $T = S$ o $T = S \triangle \{i\}$.

Demostración. Si $T = S$ o $T = S \triangle \{i\}$, entonces S y T tienen los mismos elementos distintos de i , luego $S \sim_i T$. Recíprocamente, sea $S \sim_i T$. Si i pertenece a los dos conjuntos o a ninguno, entonces S y T coinciden también en i , y $T = S$. Si pertenece exactamente a uno, digamos $i \in S$ e $i \notin T$, entonces $T = T \setminus \{i\} = S \setminus \{i\} = S \triangle \{i\}$, y análogamente en el otro caso. $\square$

Observación 5.3. La relación $\sim_i$ es de equivalencia, pues es el núcleo de la aplicación $S \mapsto S \setminus \{i\}$; por el lema 5.2, además, sus clases son exactamente los pares $\{S, S \triangle \{i\}\}$. La sección 3.4 señalaba que ciertas fórmulas son válidas exactamente en los marcos con cierta propiedad. Por ejemplo, $\Box p \rightarrow p$ lo es en los reflexivos. Las tres propiedades que hacen de $\sim_i$ una equivalencia se corresponden, en ese mismo sentido, con tres esquemas:

reflexiva	(T)	$\Box \varphi \rightarrow \varphi$	lo sabido es cierto,
transitiva	(4)	$\Box \varphi \rightarrow \Box \Box \varphi$	quien sabe algo sabe que lo sabe,
euclídea	(5)	$\neg \Box \varphi \rightarrow \Box \neg \Box \varphi$	quien no sabe algo sabe que no lo sabe,

donde *euclídea* significa que de $w \sim v$ y $w \sim u$ se sigue $v \sim u$. Los dos últimos esquemas se conocen como *introspección positiva* y *negativa*, respectivamente [8, cap. 3]. Las tres propiedades son en realidad redundantes: una relación reflexiva y euclídea es simétrica (de $w \sim v$ y $w \sim w$ se sigue $v \sim w$) y también transitiva (si $w \sim v$ y $v \sim u$, la simetría da $v \sim w$ y la propiedad euclídea aplicada en v da $w \sim u$), luego es de equivalencia. **S5**, el sistema **K** ampliado con estos esquemas, suele axiomatizarse solo con (T) y (5). Este sistema es la lógica epistémica estándar. Las correspondencias entre esquemas y propiedades de los marcos que acabamos de invocar no se demuestran en este trabajo; son el objeto del capítulo 3 de [4], como ya se indicó en la sección 3.4. Tampoco usaremos **S5** como sistema, porque en este capítulo no se demuestra nada en un cálculo axiomático y todo son comprobaciones semánticas dentro de un modelo concreto.

Cerramos la sección con dos comprobaciones sencillas que sirven para convencerse de que el modelo dice lo que queremos.

- Cada niño conoce el estado de los demás: si $i \neq j$, las fórmulas $m_j \rightarrow \Box m_j$ y $\neg m_j \rightarrow \Box \neg m_j$ son válidas en $M^{(i)}$. Para la primera, de $j \in S$ y $S \sim_i T$ se sigue $j \in S \setminus \{i\} = T \setminus \{i\} \subseteq T$, donde el primer paso usa que $j \neq i$. Para la segunda, de $j \notin S$ y $S \sim_i T$ se sigue $j \notin T$, pues $j \in T$ daría $j \in T \setminus \{i\} = S \setminus \{i\} \subseteq S$.
- Ningún niño sabe nada sobre su propia frente: para todo mundo S y todo i se tiene $M^{(i)}, S \not\models \Box m_i$ y $M^{(i)}, S \not\models \Box \neg m_i$, porque S y $S \triangle \{i\}$ son dos mundos indistinguibles para i , uno con m_i verdadera y otro con m_i falsa. Dicho con el operador dual, $M^{(i)}, S \models \Diamond m_i \wedge \Diamond \neg m_i$: el niño considera posibles las dos cosas.

5.4. Cómo los anuncios públicos cambian el modelo

El modelo describe la situación inicial, pero el acertijo es dinámico. Los anuncios del adulto y las respuestas de los niños son sucesos que cambian lo que cada niño sabe.

Definición 5.4 (Restricción). Sea $M = (W, R, V)$ un modelo de Kripke y sea $P \subseteq W$ no vacío. El modelo *restringido* $M|_P$ tiene como mundos los de P , como relación la restricción $R \cap (P \times P)$ y como valoración la restricción de V a P .

La condición $P \neq \emptyset$ es la que exige la definición 3.8, y en el acertijo se cumplirá siempre.

Si algo se anuncia en voz alta y es cierto, todos descartan a la vez los estados incompatibles con lo anunciado, y nadie gana ni pierde capacidad de observación, de modo que quien no distinguía dos mundos supervivientes los sigue sin distinguir. Esto define qué entendemos por anuncio público: cuando P es el conjunto de mundos donde una fórmula φ es verdadera, $M|_P$ es el modelo actualizado tras anunciar φ , y es el operador básico de la *lógica de anuncios públicos* de Plaza [14] (véase [16] para un tratamiento moderno). La restricción se aplica a la vez a los n modelos, porque el anuncio lo oyen todos. Nótese que la definición 5.4 admite cualquier P no vacío, sin exigir que sea el conjunto de verdad de una fórmula; esto será importante enseguida.

Veamos qué anuncios hay en el acertijo. El primero es el del adulto, “al menos uno de vosotros está manchado”, que es verdadero exactamente en los mundos S con $|S| \geq 1$. Es decir, que elimina un solo mundo, el $\emptyset$. Los siguientes anuncios son las respuestas. Como todos responden a la vez y en voz alta, que nadie diga nada equivale a un único anuncio público, “ningún niño sabe si está manchado”; y ese anuncio elimina, por definición, los mundos en los que algún niño sí lo sabría. El trabajo consiste en identificar los modelos que van apareciendo.

Observación 5.5. Ese segundo anuncio no es una fórmula de nuestro lenguaje. Escrito con todas las letras sería

$$\bigwedge_{i=1}^n (\neg \Box m_i \wedge \neg \Box \neg m_i),$$

pero cada una de sus partes ha de evaluarse en el modelo del niño correspondiente, de modo que la conjunción mezcla los n modelos y no puede formarse en un lenguaje con una sola caja (sección 5.2). Esto no impide usarla: la definición 5.4 solo necesita un subconjunto de W , y

$$P = \{ S \in W : \text{para todo } i, M^{(i)}, S \not\models \Box m_i \text{ y } M^{(i)}, S \not\models \Box \neg m_i \}$$

es un subconjunto de W perfectamente determinado, definido en el metalenguaje a partir de condiciones que sí son monoagentes. El modelo multiagente y el anuncio escrito como

fórmula son el tratamiento estándar [8, cap. 1], y la familia $(M^{(i)})_{i \in N}$ son esos mismos datos agrupados de otro modo (sección 5.2). Lo propio de este trabajo es solo la decisión de presentarlo de esta manera, que no viene impuesta por la matemática sino por el lenguaje de una sola caja que se formaliza en el capítulo 4.

Definición 5.6 (Escenarios). Para $0 \leq r \leq n$, sea

$$W_r = \{ S \subseteq N : |S| \geq r \}, \quad M_r^{(i)} = M^{(i)}|_{W_r} \quad (1 \leq i \leq n).$$

Llamaremos *escenario* r al sistema formado por los n modelos $M_r^{(1)}, \dots, M_r^{(n)}$, que comparten todos el mismo conjunto de mundos W_r . Un *mundo del escenario* r es, por tanto, un elemento de W_r , esto es, un $S \subseteq N$ con $|S| \geq r$. En particular, $W_0 = \mathcal{P}(N)$ y $M_0^{(i)} = M^{(i)}$, de modo que el escenario 0 es el cubo entero.

La cota $r \leq n$ garantiza que $W_r \neq \emptyset$, pues $N \in W_r$ siempre, como exige la definición 5.4. Obsérvese además que el mundo real sobrevive a todas las restricciones que vamos a usar: si $k = |S^*|$, entonces $S^* \in W_r$ para todo $r \leq k$. Esto es imprescindible, porque un anuncio público solo tiene sentido cuando lo anunciado es verdadero en el mundo real.

Así, $M_0^{(i)}$ es el cubo entero (todo mundo tiene al menos 0 manchados) y $M_1^{(i)}$ es el modelo tras el anuncio del adulto. El teorema de la sección siguiente demostrará que $M_r^{(i)}$ es el modelo vigente en la ronda r -ésima, es decir, el que queda tras el anuncio del adulto y $r - 1$ rondas de silencio; en él todos los mundos tienen al menos r manchados, de modo que a esas alturas ningún niño considera ya posible que haya menos de r frentes sucias. El anuncio del adulto borra del cubo de la figura 5.1 el vértice $\emptyset$ y sus tres aristas; obsérvese que, hecho eso, en un mundo de un solo manchado ese niño se queda sin alternativa y sabe que está manchado, que es la razón de que los mundos del primer nivel sean los que la primera ronda de silencio eliminará. La figura 5.2 muestra los dos escenarios siguientes.

5.5. El teorema

Fijemos primero la terminología.

Definición 5.7 (Saber si está manchado). Sean $0 \leq r \leq n$, sea S un mundo del escenario r y sea $i \in N$. Diremos que el niño i *sabe si está manchado* en S , dentro del escenario r , cuando

$$M_r^{(i)}, S \models \Box m_i \vee \Box \neg m_i,$$

esto es, cuando sabe que lo está o sabe que no lo está.

La proposición siguiente describe exactamente quién sabe qué en cada escenario. Obsérvese que sus tres apartados agotan los casos posibles, pues un mundo S del

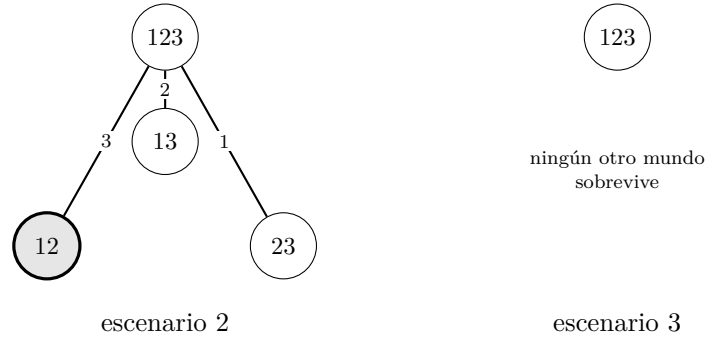


Figura 5.2: Los escenarios 2 y 3 para $n = 3$, es decir, los modelos vigentes en las rondas segunda y tercera. En el escenario 2, el mundo real $\{1, 2\}$ solo está conectado con $\{1, 2, 3\}$, y por la arista del niño 3: los niños 1 y 2 no tienen ya ninguna alternativa y saben que están manchados, mientras que el niño 3 sigue dudando entre estar limpio (mundo $\{1, 2\}$) y estar manchado (mundo $\{1, 2, 3\}$). Con $k = 2$ el juego termina aquí, en la segunda ronda; el escenario 3 solo llega a ser el modelo vigente si $k = 3$.

escenario r cumple $|S| \geq r$ y el niño i está o no está en S .

Proposición 5.8. Sean $0 \leq r \leq n$ y S un mundo del escenario r . Entonces, para cada niño i :

- (a) si $i \in S$ y $|S| = r$, entonces $M_r^{(i)}, S \models \Box m_i$; en particular, i sabe si está manchado en S ;
- (b) si $i \in S$ y $|S| > r$, entonces $M_r^{(i)}, S \not\models \Box m_i \vee \Box \neg m_i$;
- (c) si $i \notin S$, entonces $M_r^{(i)}, S \not\models \Box m_i \vee \Box \neg m_i$.

En los casos (b) y (c), por tanto, el niño i no sabe si está manchado en S .

Demostración. Por el lema 5.2, los únicos mundos indistinguibles de S para el niño i son S y $S \triangle \{i\}$; toda la demostración consiste en ver cuál de los dos sobrevive en el escenario r .

(a) Sea T un mundo del escenario con $S \sim_i T$ y supongamos, por reducción al absurdo, que $i \notin T$. Como $i \in S$, no puede ser $T = S$, luego $T = S \setminus \{i\}$ y $|T| = |S| - 1 = r - 1$. Pero T está en el escenario r , así que $|T| \geq r$: contradicción (nótese que $r \geq 1$, porque S contiene a i y $|S| = r$). Por tanto $i \in T$ para todo mundo accesible, es decir, $M_r^{(i)}, S \models \Box m_i$.

(b) El mundo $T = S \setminus \{i\}$ cumple $|T| = |S| - 1 \geq r$, porque $|S| > r$, luego sigue en el escenario, e $i \notin T$. Como también S está en el escenario e $i \in S$, el niño i considera posibles dos mundos que discrepan sobre m_i : en T falla m_i , lo que refuta $\Box m_i$, y en S falla $\neg m_i$, lo que refuta $\Box \neg m_i$.

(c) El mundo $T = S \cup \{i\}$ cumple $|T| \geq |S| \geq r$, luego sigue en el escenario, e $i \in T$. De nuevo S y T discrepan sobre m_i y ninguno de los dos ha sido eliminado, así que i no sabe nada sobre su propia frente. $\square$

El apartado (c) permite extraer una observación. Un niño limpio nunca averigua que lo está a partir de los anuncios de que nadie sabe nada, por muchas rondas de silencio que pasen. La razón es que el mundo alternativo que él considera posible tiene un manchado *más* que el real, y esos anuncios sucesivos no eliminan dichos mundos.²

Corolario 5.9. *Sean $1 \leq r \leq n$ y S un mundo del escenario r . Entonces algún niño sabe si está manchado en S si y solo si $|S| = r$. En consecuencia, si $r < n$, los mundos del escenario r que sobreviven al anuncio “ningún niño sabe si está manchado” son exactamente los del escenario $r + 1$.*

Demostración. Si $|S| = r$, como $r \geq 1$ el conjunto S no es vacío; tomando $i \in S$, el apartado (a) da $M_r^{(i)}, S \models \Box m_i$. Recíprocamente, si $|S| \neq r$, entonces $|S| > r$ porque S está en el escenario r , y los apartados (b) y (c) cubren todos los niños: ninguno sabe nada. Para la segunda parte, sobreviven los mundos con $|S| \geq r$ en los que nadie sabe nada, que son los que cumplen además $|S| \neq r$, es decir, $|S| \geq r + 1$; y como las relaciones y la valoración son en ambos casos las de $M^{(i)}$ restringidas, los modelos coinciden. $\square$

Con esto, el acertijo está resuelto: solo queda encadenar las rondas.

Teorema 5.10 (Los niños embarrados). *Supongamos que el mundo real es S^* con $k = |S^*|$ y $1 \leq k \leq n$. Entonces, para cada r con $1 \leq r \leq k$, el modelo vigente en la ronda r es el escenario r y:*

- *si $r < k$, ningún niño sabe si está manchado, y todos guardan silencio;*
- *si $r = k$, los niños de S^* saben que están manchados y contestan que sí, y los demás siguen sin saber nada y callan.*

Demostración. Sea $P(r)$ la afirmación “el modelo vigente en la ronda r es el escenario r ”. Demostramos $P(r)$ para $1 \leq r \leq k$ por inducción sobre r , y de camino obtenemos las dos viñetas.

Caso base. El modelo vigente antes de que nadie hable es el escenario 0, el cubo entero. El anuncio del adulto es cierto en S^* , pues $k \geq 1$, y su conjunto de verdad es

²Cosa distinta es lo que ocurre *después* de la ronda k . El niño limpio i habrá oído entonces que los niños de S^* sí saben, y ese anuncio elimina el mundo $S^* \cup \{i\}$: en él hay $k + 1$ manchados y, por los apartados (b) y (c), nadie habría hablado. Al niño i solo le quedaría el mundo S^* y sabría que está limpio. Lo que demuestra el apartado (c) es, precisamente, que ningún anuncio de la forma “nadie lo sabe” le sirve de nada.

W_1 , de modo que la restricción da el escenario 1: este es el modelo con el que los niños responden a la primera pregunta. Luego $P(1)$.

Paso inductivo. Sean $1 \leq r < k$ y supongamos $P(r)$. El mundo real cumple $|S^*| = k > r$, así que por los apartados (b) y (c) de la proposición 5.8 ningún niño sabe si está manchado y todos guardan silencio; esta es la primera viñeta. Ese silencio conjunto es el anuncio “ningún niño sabe si está manchado”, cierto en S^* , y la segunda parte del corolario 5.9 identifica el modelo restringido con el escenario $r + 1$, que es por tanto el modelo vigente en la ronda $r + 1$. Luego $P(r + 1)$.

Conclusión. Por $P(k)$, el modelo vigente en la ronda k es el escenario k , y $|S^*| = k$. El apartado (a) de la proposición 5.8 da $M_k^{(i)}, S^* \models \Box m_i$ para todo $i \in S^*$, y el apartado (c), que ningún niño limpio sabe nada. Esta es la segunda viñeta. $\square$

5.6. El anuncio del adulto y el conocimiento común

Volvamos a la paradoja que dejamos planteada en la sección 5.1. Si hay dos o más niños manchados, todos ven al menos una frente sucia, de modo que todos saben ya lo que el adulto va a decirles antes de que abra la boca. Y sin embargo, sin ese anuncio el acertijo no se resuelve. ¿Qué puede aportar una frase que todo el mundo sabe?

La respuesta corta es que el anuncio no informa sobre el barro. Informa sobre lo que los demás saben.

Para verlo, situémonos *antes* de que el adulto hable por primera vez. Tomemos $n = 3$ y el mundo real $S^* = \{1, 2\}$, el que aparece sombreado en la figura 5.1, y llamemos φ al enunciado “al menos uno de vosotros está manchado”, que es verdadero en todos los mundos del cubo salvo en $\emptyset$, que todavía no se ha dicho.

Empecemos preguntando si todos saben φ . La respuesta es que sí, y cada uno por su cuenta: el niño 1 lo sabe porque ve la frente sucia del 2, el niño 2 porque ve la del 1, y el niño 3 porque ve las dos. Dicho con los mundos que cada uno considera posibles: el niño 1 duda entre $\{1, 2\}$ y $\{2\}$, y en ambos hay alguien manchado; el niño 2 duda entre $\{1, 2\}$ y $\{1\}$, ídem; el niño 3, entre $\{1, 2\}$ y $\{1, 2, 3\}$, ídem. Ninguno de los tres considera posible que no haya nadie con la frente sucia.

Preguntemos ahora si todos saben que todos lo saben, y aquí la respuesta es que no. Pensemos en el niño 1: ¿por qué habría de saber el niño 2 que hay alguien manchado? Porque le ve a él. Pero el niño 1 no se ve la propia frente y no ha descartado el mundo $\{2\}$, en el que él estaría limpio. Y si el mundo fuera $\{2\}$, el niño 2 no vería ninguna frente sucia y no habría descartado $\emptyset$, donde φ es falsa. El niño 1 no puede, por tanto, descartar que el niño 2 no sepa φ .

Esto puede parecer extraño. El niño 1 ve perfectamente la frente sucia del 2 y sabe de sobra que el 2 ve la suya. Nadie cree que no haya ningún manchado. ¿Cómo puede

entonces fallar el “segundo nivel de conocimiento”? Lo que falla no es una creencia de nadie, sino una cadena de hipótesis encadenadas: el niño 1 no descarta un mundo en el que el niño 2 no descartaría un mundo en el que φ sería falsa. En el cubo de la figura 5.1 esa cadena se ve dibujada, y es el camino de dos aristas

$$\{1, 2\} \sim_1 \{2\} \sim_2 \emptyset.$$

Que ese camino atravesase mundos que nadie se cree de verdad es irrelevante. La semántica de Kripke dice que un niño sabe lo que es cierto en todos los mundos que no ha descartado, y el niño 1 no ha descartado $\{2\}$.

Escribimos $E\varphi$ para “todos saben φ ”, $E^2\varphi$ para “todos saben que todos saben φ ”, y así sucesivamente: cada exponente es un peldaño más de una escalera de conocimiento anidado. A $E^i\varphi$ le llamamos *conocimiento mutuo de nivel i* .

Llamamos *conocimiento común* de φ , y lo escribimos $C\varphi$, a disponer de la escalera entera, es decir, a que se cumplan todos los $E^j\varphi$ a la vez, *ad infinitum*³. Por lo que acabamos de ver, antes del anuncio, el conocimiento común falla siempre, por muchos niños manchados que haya.

¿Y qué hace entonces el anuncio público? Elimina el mundo $\emptyset$ del modelo. Y una vez fuera del modelo ya no hay ningún camino, de ninguna longitud, que lleve a un mundo donde φ sea falsa, porque ese mundo ha dejado de existir. La escalera de conocimiento aparece entera directamente, no como una construcción de peldaños anidados.

Todo este razonamiento mezcla el conocimiento de varios niños en una misma fórmula, empezando por el “el niño 1 no sabe si el niño 2 sabe” con el que abríamos la sección. Queda por tanto fuera del lenguaje con el que estamos trabajando, igual que ocurría con el anuncio de la observación 5.5, y por eso lo hemos discutido de manera informal, sobre el dibujo del cubo, en lugar de escribirlo como fórmulas. Es la consecuencia de no usar un modelo multiagente, que es lo habitual en el tratamiento de estos problemas, como comentamos en la sección 5.2.

En realidad, esta es la razón de que este capítulo se haya escrito y modelizado en este lenguaje aunque el acertijo no lo exija y pueda “resolverse” informalmente. “El niño 1 imagina que el niño 2 podría imaginar. . .” no es algo que se pueda escribir en un asistente de demostración. Un conjunto de mundos, una relación y una recursión sobre fórmulas, sí y son exactamente los objetos que el capítulo 4 ya tiene definidos y verificados en Lean. La modelización de este capítulo deja el acertijo a un paso de ser código, con la salvedad de que ese paso no se ha dado en este trabajo, por las razones que se comentan en el capítulo 6.

³Esto no es una definición formal para lógica epistémica puesto que los lenguajes habituales de la misma son finitos y la fórmula $C\varphi \iff \bigwedge_{i=0}^{\infty} E^i\varphi$ no es una fórmula bien formada.

Capítulo 6

Conclusiones

El objetivo general de este trabajo era formalizar en Lean 4 la lógica modal **K** y demostrar en ese entorno su corrección y completitud respecto de los modelos de Kripke. Se desarrollan en este capítulo las conclusiones de realizar dicho proceso.

6.1. Aprendizaje de Lean, lógica modal y formalización.

La circunstancia que más ha condicionado el trabajo es que partía de cero en las dos partes que componían la propuesta. No conocía Lean ni había estudiado lógica modal, de modo que había que formalizar un contenido desconocido con una herramienta desconocida. Sin embargo, a pesar de que ha sido un proceso complicado, también ha existido cierta sinergia en el aprendizaje. Una demostración escrita por quien está aprendiendo un tema no avisa de las hipótesis que faltan o la totalidad de lo que se debe probar, y quien la escribe carece precisamente del criterio necesario para detectarlos. Con Lean, el InfoView dice en todo momento y sin ambigüedad qué se tiene y qué falta, y no deja avanzar mientras falte algo. A su vez, tener que adaptar y escribir las demostraciones de los teoremas obligaba a adquirir un buen nivel de comprensión.

De Lean, lo más costoso no fueron las tácticas, que se aprenden con los materiales de la comunidad [1, 2, 5] y con la práctica, sino adaptarse a la teoría de tipos y al grado de formalidad y rigidez que requiere un asistente de demostración. De la lógica modal, lo más reseñable ha sido estudiarla primero leyendo [4] y teniendo que adaptar dicho conocimiento a los requisitos del código. Dicha adaptación ha producido buena parte de las observaciones del capítulo 3: que el axioma (Dual) sobra en cuanto se toma $\Box$ como primitivo, que la necesidad debe quedar fuera de la deducción desde hipótesis (observación 3.19), o que la opacidad de la caja en el lema puente 3.17 es una hipótesis de inducción que no se usa. También algunos cambios en la forma de definir los términos, como el caso de las tautologías en 3.2.

Otro aprendizaje relevante es que el esfuerzo de formalizar no es proporcional a la profundidad matemática. El lema puente 3.17 es un ejercicio en [4] y concentra casi todo el trabajo de la corrección, mientras que el teorema de corrección, que es el resultado con nombre, ocupa cuatro líneas.

Lo que ha quedado patente es que los asistentes de demostración son una forma distinta y poco habitual para mí de hacer matemáticas.

6.2. Reflexiones sobre Lean y la mecanización de las demostraciones

La introducción presentaba un ideal atractivo y reconocidamente ingenuo, el de un “árbol de las matemáticas” verificado y que puede recorrerse en dos direcciones con dos implicaciones distintas. Una dirección descendente, de las matemáticas más avanzadas a las más básicas, que permitiría construir un grafo de dependencias de cada resultado matemático. Otra dirección ascendente, desde las bases axiomáticas de las matemáticas hacia problemas abiertos en la actualidad, que permitiría explorar la ayuda mecánica para crear nuevas matemáticas. Se señalaba ya entonces que este idealismo tenía cierta base pero enfrentaba problemas reales también.

Para emprezar, Lean verifica que los teoremas se siguen de las definiciones, no que las definiciones digan lo que queríamos decir. Si una cláusula de `Satisfies` estuviera mal escrita, el fichero compilaría igual y el teorema final seguiría siendo cierto, solo que sobre otra lógica. Contra eso no hay verificación posible, y por eso lemas como `satisfies_neg` o `k_consistent` funcionan como control sobre el significado de lo definido. Admitir marcos con el tipo de mundos vacío (observación 4.1) es todavía más ilustrativo: una decisión que no cabe verificar, solo argumentar.

El segundo límite es económico. Lo que en [4] ocupa unas pocas páginas y un par de ejercicios ha necesitado aquí setecientas líneas, y buena parte de ellas no contiene matemáticas sino administración con subtipos que empaquetar y desempaquetar, argumentos implícitos, nombres de lemas auxiliares, etc. Las enormes diferencias entre página de manuales y líneas de código siguen siendo el obstáculo principal, y explica por qué las bibliotecas formalizadas como Mathlib tienen una extensión limitada.

De las dos direcciones del ideal, la descendente es la que hoy está al alcance: el grafo de dependencias existe y se puede consultar. Conviene precisar, sin embargo, qué ese grafo es un registro histórico y no una descripción ontológica. Es decir, recoge el camino que efectivamente se recorrió, no lo que el teorema necesita. Lean certifica que nuestra demostración se sigue de esas premisas, pero no dice que hagan falta, ni que sean las mínimas, ni que no existan rutas más cortas. Son un ejemplo las ocho tautologías de la

tabla 4.1, que son las que este desarrollo necesitó y no una base mínima. Establecer que una premisa es imprescindible exige un resultado de independencia, esto es, un teorema nuevo y por lo general más difícil, y no un subproducto de la verificación.

La dirección ascendente es, de momento, una idea muy limitada. Las aplicaciones más prometedoras en el futuro cercano probablemente vendrán del hecho de que comprobar una demostración candidata sea muy barato, con independencia de su procedencia. Esto aplica especialmente a herramientas de inteligencia artificial, que destacan por su falta de fiabilidad. La razón sensata para el optimismo no es que las máquinas vayan a encontrar las demostraciones, sino que verificarlas no sea caro.

6.3. Mejoras y ampliaciones posibles

El desarrollo deja abiertas varias continuaciones posibles:

- **Complejidad fuerte.** Definir la consecuencia semántica $\Sigma \models \varphi$ y comprobar que el mundo que devuelve Lindenbaum satisface las premisas de Σ . El modelo canónico y los lemas se reutilizan sin cambios.
- **Lógicas modales más fuertes.** Añadiendo axiomas a **K** se obtienen **T**, **S4**, **S5**, y con ellos la correspondencia entre cada axioma y una propiedad del marco, donde encajaría `ValidInFrame`.
- **Niños embarrados en Lean.** El capítulo 5 deja el acertijo modelizado con los mismos objetos que el capítulo 4 ya tiene verificados, pero no llega a escribirlo en Lean. El obstáculo está en el enunciado del teorema 5.10: “el modelo vigente en la ronda r ” es una noción narrada, no definida, y formalizarlo obligaría a definir el protocolo de rondas como un objeto y no como un “relato”.
- **Multimodalidad y lógica epistémica.** Indexar la caja por agentes, $\Box_i$, y añadir conocimiento distribuido y común para poder desarrollar más en detalle lógica epistémica, incluyendo la lógica de anuncios públicos de Plaza [14] y desarrollos posteriores.

Bibliografía

- [1] J. Avigad, L. de Moura, S. Kong, and S. Ullrich. Theorem proving in Lean 4. https://lean-lang.org/theorem_proving_in_lean4/, 2024. Consultado en 2026.
- [2] J. Avigad and P. Massot. Mathematics in Lean. https://leanprover-community.github.io/mathematics_in_lean/, 2024. Consultado en 2026.
- [3] H. Barendregt and F. Wiedijk. The challenge of computer mathematics. *Philosophical Transactions of the Royal Society A*, 363(1835):2351–2375, 2005.
- [4] P. Blackburn, M. de Rijke, and Y. Venema. *Modal Logic*, volume 53 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press, 2001.
- [5] K. Buzzard et al. The natural number game (Lean 4). <https://adam.math.hhu.de/>, 2023. Consultado en 2026.
- [6] N. G. de Bruijn. The mathematical language AUTOMATH, its usage, and some of its extensions. In M. Laudet, D. Lacombe, L. Nolin, and M. Schützenberger, editors, *Symposium on Automatic Demonstration (Versailles, diciembre de 1968)*, volume 125 of *Lecture Notes in Mathematics*, pages 29–61. Springer, Berlín, 1970.
- [7] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE 28*, volume 12699 of *Lecture Notes in Computer Science*. Springer, 2021.
- [8] R. Fagin, J. Y. Halpern, Y. Moses, and M. Y. Vardi. *Reasoning About Knowledge*. MIT Press, Cambridge, MA, 1995. Edición revisada, 2003.
- [9] A. Grabowski, A. Kornilowicz, and A. Naumowicz. Mizar in a nutshell. *Journal of Formalized Reasoning*, 3(2):153–245, 2010.
- [10] S. Kripke. Semantical analysis of modal logic I: Normal modal propositional calculi. *Zeitschrift für mathematische Logik und Grundlagen der Mathematik*, 9:67–96, 1963.

- [11] J. E. Littlewood. *A Mathematician's Miscellany*. Methuen, Londres, 1953.
- [12] J. Martinena Ceba. Lógica modal en Lean. <https://github.com/Jorbol/tfg-logica-modal-lean>, 2026.
- [13] M. Noguchi et al. Foundation: Formalization of mathematical logic in Lean 4. <https://github.com/FormalizedFormalLogic/Foundation>, 2026. Consultado en 2026.
- [14] J. Plaza. Logics of public communications. In *Proceedings of the 4th International Symposium on Methodologies for Intelligent Systems*, pages 201–216, 1989. Reimpreso en *Synthese* 158(2):165–179, 2007.
- [15] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. ACM, 2020.
- [16] H. van Ditmarsch, W. van der Hoek, and B. Kooi. *Dynamic Epistemic Logic*, volume 337 of *Synthese Library*. Springer, Dordrecht, 2008.
- [17] M. Wu and R. Goré. Verified decision procedures for modal logics. In *10th International Conference on Interactive Theorem Proving (ITP 2019)*, volume 141 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:19, Dagstuhl, 2019. Schloss Dagstuhl–Leibniz-Zentrum für Informatik.